

Part 5 Section 2: Correlation and Linear Models

Aaron Robotham

Libraries needed for this chapter: magicaxis, foreach, MASS.

```
library(magicaxis, quietly=TRUE)
library(foreach, quietly=TRUE)
library(MASS, quietly=TRUE)
```

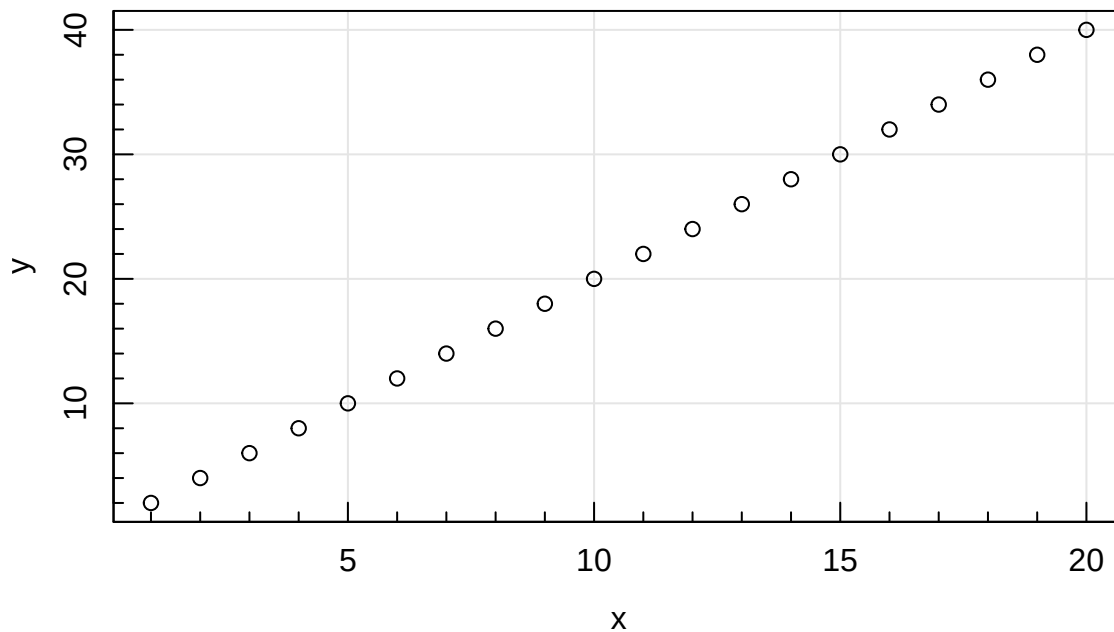
So everybody running this chapter gets the same outputs, we should first set the seed.

```
set.seed(1)
```

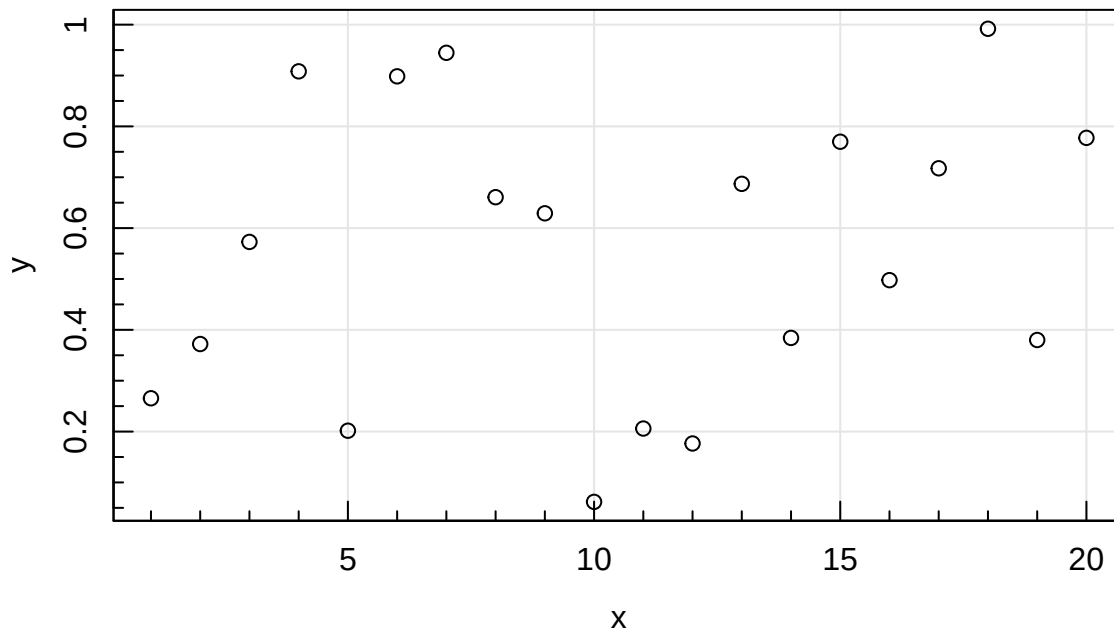
Linear Correlation

You probably have a good idea of what we mean by correlation, since it is everyday term. I.e. without deep thought you can probably order the degree of correlation between the abscissa (x-axis) and ordinate (y-axis) in these three plots.

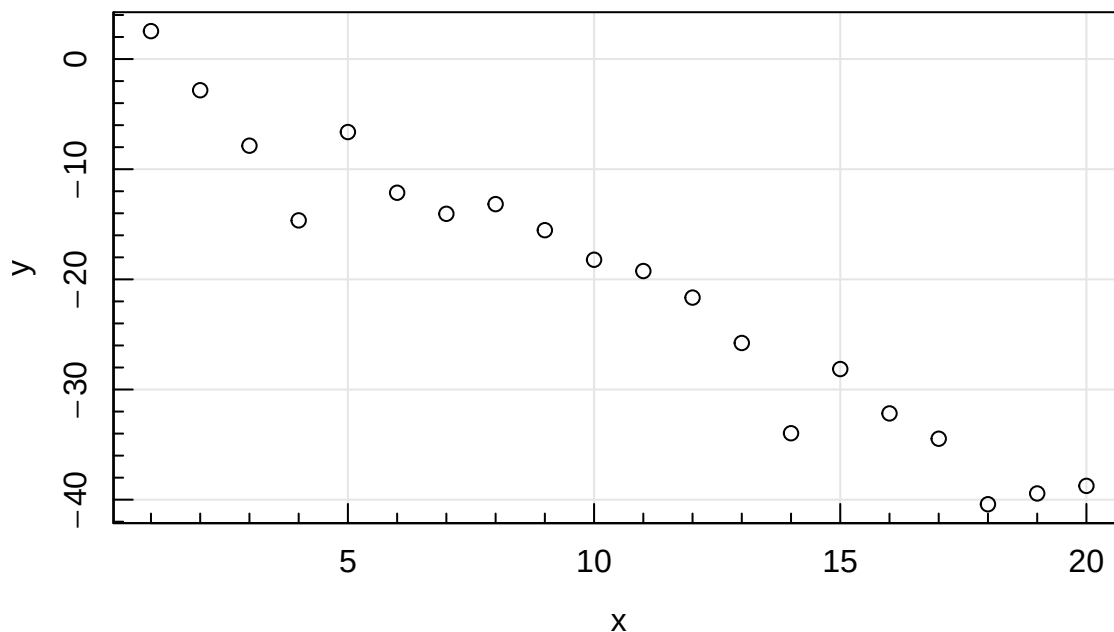
```
magplot(1:20, 1:20*2, xlab='x', ylab='y')
```



```
temp_unif = runif(20)
magplot(1:20, temp_unif, xlab='x', ylab='y')
```



```
y_norm = -(1:20)*2 + rnorm(20,sd=3)
magplot(1:20, y_norm, xlab='x', ylab='y')
```



However, we are quantitative statisticians (or getting there), so we would like to put a more definite number on how correlated these plots are. Plotting 2D data and commenting on how related the two axes are is pretty much what 80% of academia publications are about after all.

Pearson Correlation Coefficient

By far the most popular way of parameterising the correlation in a 2D plane is the Pearson correlation coefficient. This is defined as the following:

$$\text{cor}(x, y) = \frac{\text{cov}(x, y)}{\sigma(x)\sigma(y)}$$

Where $\text{cov}(x, y)$ is the covariance between x and y , and σ is the standard deviation. cor will always have a value between -1 and 1. We can apply it to the above toy distributions:

```
cov(1:20, 1:20*2)/(sd(1:20)*sd(1:20*2))
```

```
## [1] 1
```

```
cov(1:20, temp_unif)/(sd(1:20)*sd(temp_unif))
```

```
## [1] 0.1673181
```

```
cov(1:20, y_norm)/(sd(1:20)*sd(y_norm))
```

```
## [1] -0.972981
```

So a value towards 1 means positively correlated, and towards -1 means negatively correlated. A value near 0 means little evidence of correlation. **R** has functions **cor** or **cor.test** for computing this directly. The former only computes the coefficient of interest, whilst the latter computes a number of additional characteristics.

```
cor(1:20, y_norm)
```

```
## [1] -0.972981
```

```
cor.test(1:20, y_norm, method='pearson')
```

```
##
```

```
## Pearson's product-moment correlation
```

```
##
```

```
## data: 1:20 and y_norm
```

```
## t = -17.879, df = 18, p-value = 6.608e-13
```

```
## alternative hypothesis: true correlation is not equal to 0
```

```
## 95 percent confidence interval:
```

```
## -0.9894709 -0.9315543
```

```
## sample estimates:
```

```
## cor
```

```
## -0.972981
```

In linear correlation/regression analysis it is typical to denote $r^2 = \text{cor}(x, y)^2$, where this quantity is known as the coefficient of determination and scales between 0 and 1 (but is usually called ‘r-squared’ when talking about it). This is discussed more later. In the above $\text{cor}(x, y)$ is output by the list element *estimate*, so we can now compute r^2 easily:

```
cor(1:20, y_norm)^2
```

```
## [1] 0.9466921
```

```
cor.test(1:20, y_norm, method='pearson')$estimate^2
```

```
## cor
```

```
## 0.9466921
```

Error Free Linear Regression

In the scenario that the x and y data have no error associated with their measurements, we can calculate the linear function that minimises the amount of variance along the y direction (often called y on x regression):

$$y = mx + b \text{ where}$$

$$m = \frac{\text{cov}(x, y)}{\text{var}(x)}$$

$$b = \bar{y} - m\bar{x}$$

Using this formula we can determine the line that minimises the variance along y

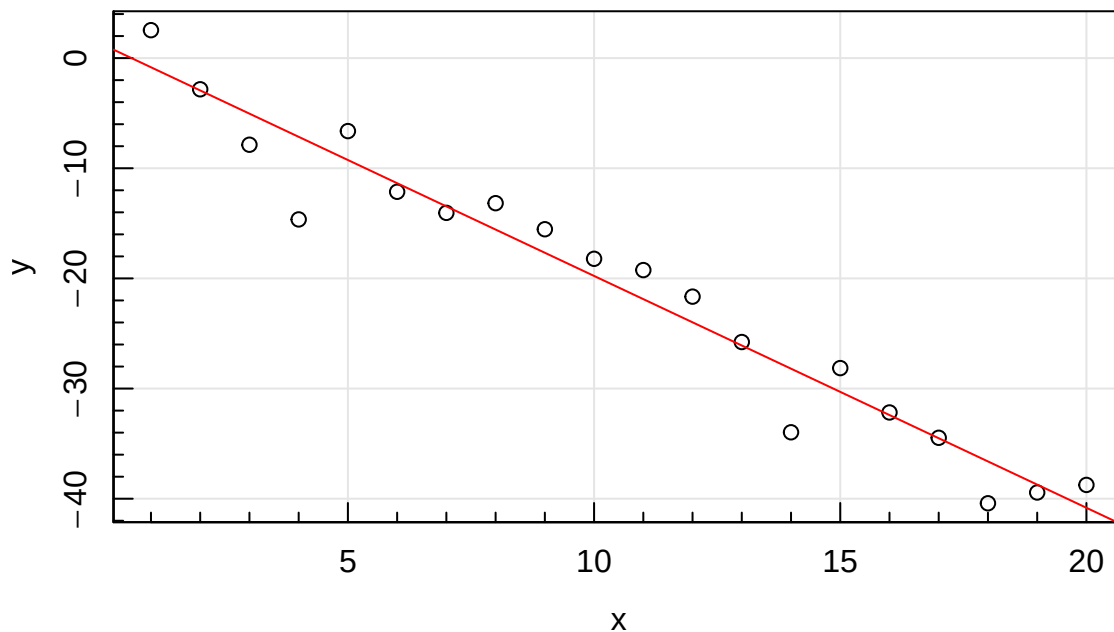
```
m = cov(1:20, y_norm)/var(1:20)
m
```

```
## [1] -2.104831
```

```
b = mean(y_norm) - m*mean(1:20)
b
```

```
## [1] 1.273487
```

```
magplot(1:20, y_norm, xlab='x', ylab='y')
abline(b,m,col='red')
```



R has a fully featured function **lm** that can be used to do very sophisticated linear modelling of multi-dimensional data, however in its simplest form we can use it to replicate our 2D y on x regression:

```
x = 1:20
y = y_norm
out = lm(formula = y ~ x)
print(out)
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Coefficients:
## (Intercept)          x
##      1.273      -2.105
```

```
summary(out)
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.4983 -0.7319  0.2742  2.2016  3.3667
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)    1.2735      1.4103    0.903    0.378
## x             -2.1048      0.1177 -17.879 6.61e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.036 on 18 degrees of freedom
## Multiple R-squared:  0.9467, Adjusted R-squared:  0.9437
## F-statistic: 319.7 on 1 and 18 DF,  p-value: 6.608e-13
```

We can see it has found the same values for our slope (m) and intercept (b). It also computes a whole host of other statistics (e.g. uncertainties on these quantities), including ‘r-squared’ (i.e. r^2). This is the squared Pearson correlation coefficient from above.

```
cor(1:20, y_norm)^2
```

```
## [1] 0.9466921
```

Now we have our linear model it is easy to see what r^2 really represents. We first compute the residuals of our y data minus the linear model:

```
y_resid = (y_norm) - 1:20*m + b
```

And then we can compute the ratio of the residual data compared to the original:

```
var(y_resid)/var(y_norm)
```

```
## [1] 0.05330789
```

So this is the fraction of initial variance remaining, 1 minus this is the amount *explained* by our linear model:

```
1 - var(y_resid)/var(y_norm)
```

```
## [1] 0.9466921
```

I.e. this is our r^2 statistic, and it represents the fraction of y variance explained by our linear model. This concept can be generalised for multi-dimensional non-linear models to say what fraction of the dependent dimension variance can be explained by a given model (usually written as R^2). This concept is discussed later.

A Brief Note on R Formula

You will notice above that we use a *formula* when inputting arguments to **lm**. **R** formula are extremely complex and sophisticated, and allow highly generalised ways of abstracting relationships between parameters or interaction groups. In Python you can use the **statsmodels.formula.api** module for similar effect (which is based on **R** formula of course). It is beyond the scope of this course to discuss it in detail, but here we will go over some basics.

For simple modelling the main parts are the dependent quantity (LHS) and the independent or predictor (RHS). The tilde symbol “~” tells **R** how to determine the LHS or RHS, since you can have multiple arguments on each.

For a simple linear model for a line this means our model looks like

```
y ~ x
```

But we could also fit a 3D plane with a model that looks like:

```
z ~ x + y
```

The plus here means we are telling **R** that our linear model will be a linear mix of the x and y terms with unknown scaling constants.

lm can also fit a model that is a generic linear mixture of almost any terms, e.g. x , x^2 , y and $\ln y$. To do this we have to isolate the some terms with **I**:

```
z ~ x + I(x^2) + y + log(y)
```

This should give you the basics of how we specify simple **R** models, but there is a lot of material available in **R** help and online if you need to know more. A key takeaway is that in this context ‘linear’ does not simply mean a straight line since any polynomial form can be re-parameterised to create such a line/plane.

For convenience **R** also includes its own generic polynomial creation function **poly**. This makes it easier to experiment with higher order fits without lots of error prone writing, for instance to fit terms up to and including a cubic we can write:

```
z ~ poly(x, degree=3)
```

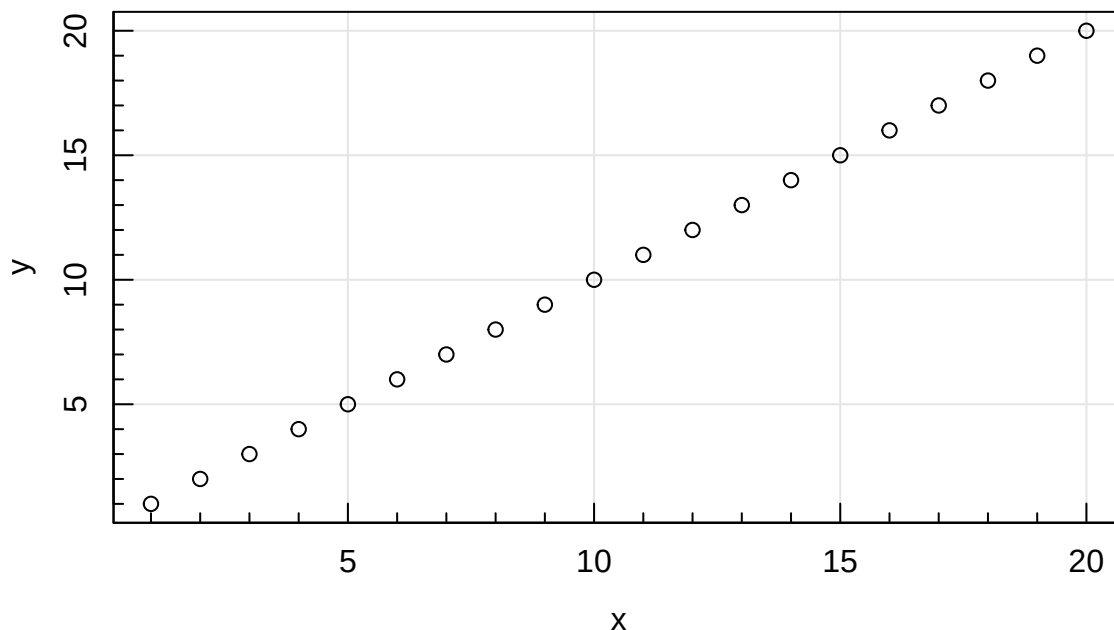
I use this method to fit complex sky models in my source extraction code **ProFound**, which we discuss more in the next chapter.

Non-Linear Correlation

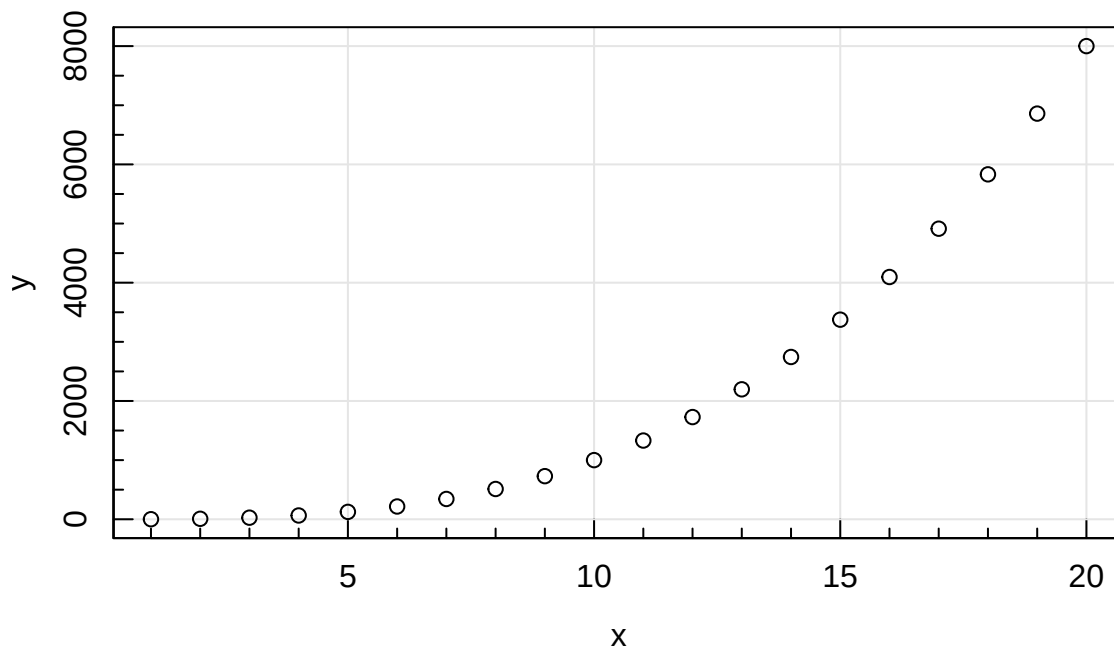
Spearman Rank Correlation

A concept closely related to the above r^2 is the Spearman rank correlation (usually denoted as ρ). Rather than using the values of x and y to determine the degree of correlation present, it instead rank orders both axes and compares the ranks using the Pearson correlation coefficient. This has the effect of measuring whether there is a strictly monotonic relationship between the two dimensions. E.g. consider

```
magplot(1:20, 1:20, xlab='x', ylab='y')
```

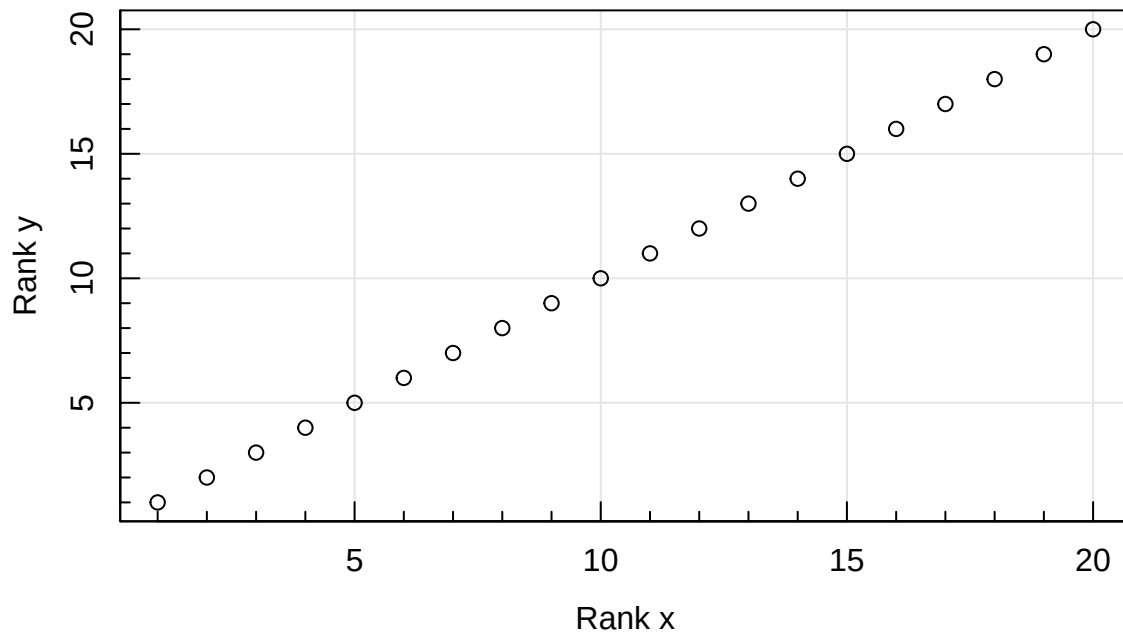


```
magplot(1:20, (1:20)^3, xlab='x', ylab='y')
```



The first is clearly linear, and the second is not (in the straight line sense), but both have y strictly increasing with x , i.e.:

```
magplot(order(1:20), order(1:20), xlab='Rank x', ylab='Rank y')
```



```
magplot(order(1:20), order((1:20)^3), xlab='Rank x', ylab='Rank y')
```

We can compute the Spearman rank correlation easily ourselves using the **order** function (which gives the location of every element as ranked within its vector). Using the x and y data from above (which remember, were negatively correlated when we last modified them):

```
cor(order(x), order(y))
```

```
## [1] -0.9699248
```

For convenience **R** can compute this more directly using the **cor.test** function with the method set to 'spearman'.

```
cor.test(x, y, method='spearman')
```

```
##
## Spearman's rank correlation rho
##
## data: x and y
## S = 2620, p-value = 6.568e-06
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## -0.9699248
```

The key thing is the Spearman correlation is robust to the shape of the relationship, as long as the monotonic nature of the relationship is maintained:

```
cor.test(x, y^3, method='spearman')
```

```
##
## Spearman's rank correlation rho
##
## data: x and y^3
## S = 2620, p-value = 6.568e-06
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## -0.9699248
```

Where as the Pearson correlation would drop notably:

```
cor.test(x, y^3, method='pearson')
```

```
##
## Pearson's product-moment correlation
##
## data: x and y^3
## t = -8.1306, df = 18, p-value = 1.944e-07
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
## -0.9545787 -0.7307113
## sample estimates:
##      cor
## -0.8865589
```

Kendall Rank Correlation

The Kendall rank correlation (τ) is also robust to the non-linear nature of the x and y relationship, again testing how monotonic (or not) it is. It is defined as:

$$\tau = \frac{(\text{con pairs}) - (\text{dis pairs})}{n(n-1)/2}$$

Where ‘con pairs’ refers to the number of concordant (x_i, y_i) (x_j, y_j) pairs (where $x_i < x_j$ and $y_i < y_j$ or $x_i > x_j$ and $y_i > y_j$), and ‘dis pairs’ refers to the number of discordant (x_i, y_i) (x_j, y_j) pairs (where $x_i < x_j$ and $y_i > y_j$ or $x_i > x_j$ and $y_i < y_j$). This is a bit of a mouthful, and code wise it means we have to loop through all $n(n-1)/2$ pair combinations, which can get expensive if not coded optimally. The direct way to calculate τ would be:

```
top = 0
for(i in 2:length(x)){
  for(j in 1:(i-1)){
    top = top + sign(x[i] - x[j]) * sign(y[i] - y[j]) #sign has value -1,0,+1
  }
}
```

```
}
top/(length(x)*(length(x) - 1)/2)
```

```
## [1] -0.8842105
```

Handily, we can also use `cor.test` to compute this for us:

```
cor.test(x, y, method='kendall')
```

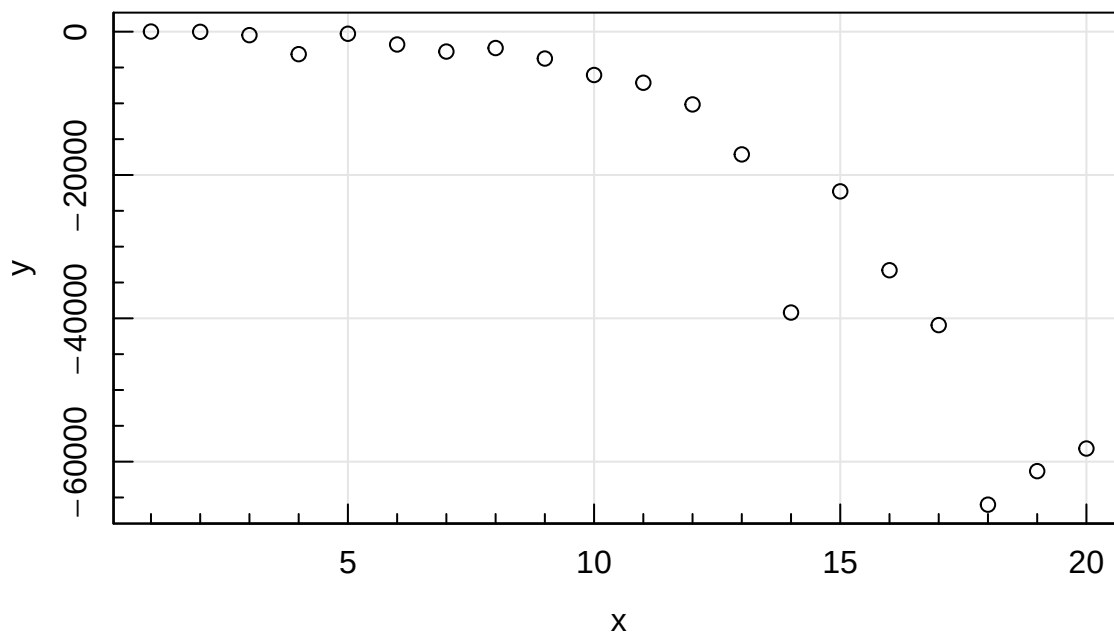
```
##
## Kendall's rank correlation tau
##
## data: x and y
## T = 11, p-value = 3.687e-11
## alternative hypothesis: true tau is not equal to 0
## sample estimates:
##      tau
## -0.8842105
```

Coefficient of Determination

As mentioned above, the concept of Pearson correlation's r^2 can be generalised to R^2 , where the model can be non-linear. In this case R^2 represents the fraction of dependent variable variance that can be explained by a model.

As a simple example we can create a cubic relationship $y_3 = y^3$.

```
y_3 = y^3
magplot(x, y_3, xlab='x', ylab='y')
```



One way to attempt to model this would be by cubing the x axis (i.e. making $x_3 = x^3$) and fitting a traditional linear relationship:

```
x_3 = x^3
out2 = lm(formula = y_3 ~ x_3)
print(out2)
```

```
##
## Call:
## lm(formula = y_3 ~ x_3)
##
```

```
## Coefficients:
## (Intercept)      x_3
##      387.974      -8.706
```

```
summary(out2)
```

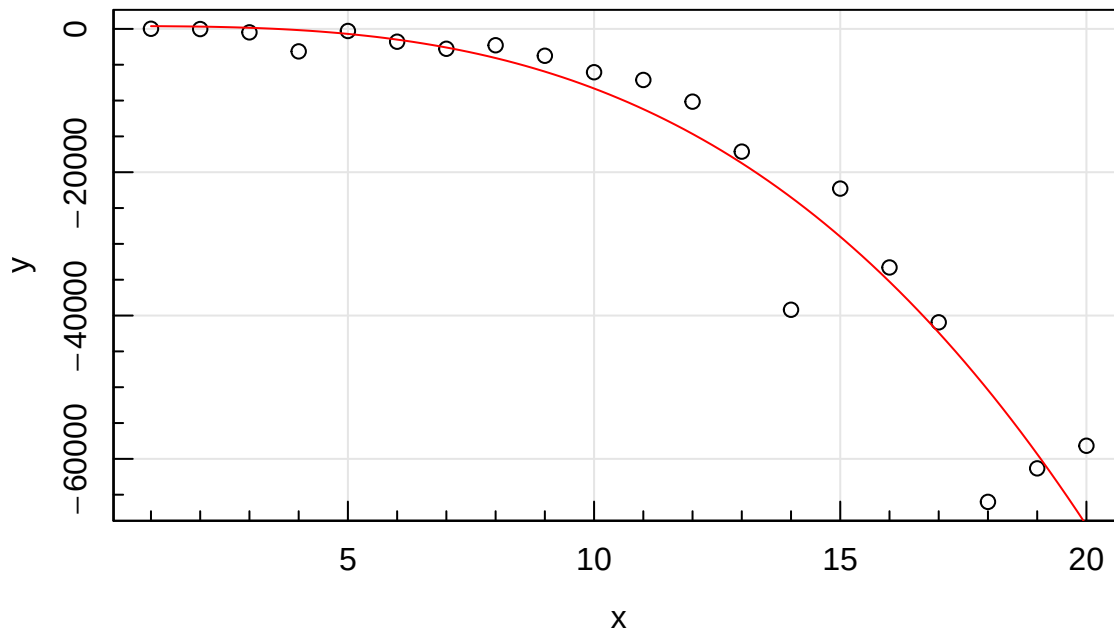
```
##
## Call:
## lm(formula = y_3 ~ x_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15692.2   -432.0    923.3   2224.2  11091.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  387.9740  1919.5237   0.202   0.842
## x_3          -8.7059    0.5835 -14.921 1.41e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6371 on 18 degrees of freedom
## Multiple R-squared:  0.9252, Adjusted R-squared:  0.921
## F-statistic: 222.6 on 1 and 18 DF,  p-value: 1.408e-11
```

As mentioned above, we can do this in the model as a convenience using **I**:

```
summary(lm(formula = I(y^3) ~ I(x^3)))
```

```
##
## Call:
## lm(formula = I(y^3) ~ I(x^3))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15692.2   -432.0    923.3   2224.2  11091.2
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  387.9740  1919.5237   0.202   0.842
## I(x^3)       -8.7059    0.5835 -14.921 1.41e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6371 on 18 degrees of freedom
## Multiple R-squared:  0.9252, Adjusted R-squared:  0.921
## F-statistic: 222.6 on 1 and 18 DF,  p-value: 1.408e-11
```

```
magplot(x, y_3, xlab='x', ylab='y')
curve(out2$coefficients[2]*x^3 + out2$coefficients[1], add=TRUE, col='red')
```



Given the above we can also compute out own R^2 directly:

```
cor(x^3, y_3)^2
```

```
## [1] 0.9251964
```

I.e. it should be clear that even though this relationship is no longer linear, our fitted model can still be thought to explain some fraction of the variance (in this case near 90%).

Mutual Information

Shannon's entropy is defined such that

$$H(X) = - \sum_{i=1}^n P(x_i) \ln P(x_i)$$

The intuition here is tricky, but it is basically telling you how predictable a certain discrete distribution is. Because probabilities are always between 0 and 1, $\ln P(x_i)$ will always be negative. Then, because of the negative sign at the front, the Shannon entropy must always be positive. The sum of all $P(x_i)$ must be 1, and the lower bound is 0 (when we only have one observation with probability 1). It is clear then that the larger the Shannon entropy is, the more random (or disordered) the system must be: lots of small probability states has more entropy than one certain state (just like in statistical mechanics).

Imagine we have a system that can have 6 states (i.e. a die) with equal probability, we would get:

```
- 6*(1/6 * log(1/6))
```

```
## [1] 1.791759
```

Now imagine instead we have a system that still has six states, but 1-4 occur with a 1/12 chance, and 5-6 with a 1/3 chance:

```
- 4*(1/12 * log(1/12)) - 2*(1/3 * log(1/3))
```

```
## [1] 1.56071
```

The entropy we get is lower, meaning the system is less random and easier to predict (it will mostly fall on 5-6). This should make sense, since indeed if we were gambling we would do best to choose the 2 sides with 1/3 chance of occurring (5-6).

We can also determine the cross entropy for two distributions:

$$H(X, Y) = - \sum_{i=1}^n \sum_{j=1}^n P(x_i, y_j) \ln P(x_i, y_j)$$

In this case imagine we have two dice. If these are unbiased then there is a 1/36 chance of any pair-wise combination being rolled;

```
- 6*6*(1/36 * log(1/36))
```

```
## [1] 3.583519
```

Which is just two times the value we got for out single unbiased die.

But imagine that in fact the two die always agree, i.e. if one shows 3 so will the other etc:

```
- 6*(1/6 * log(1/6))
```

```
## [1] 1.791759
```

This is the same as the Shannon entropy for a single unbiased die. This should make sense, since effectively we only need to know the outcome of one unbiased die to know the outcome of the other, so this is like we are only throwing on die and therefore we have the same information entropy.

This concept gets us to our final equation of interest:

$$I(X; Y) = H(X) + H(Y) - H(X, Y)$$

where $I(X; Y)$ is known as the mutual information. If this is positive then it means knowing something about the outcome of X informs us with useful information about the outcome of Y . If it is 0 then it means knowledge of X will not help us guess the outcome of Y (X and Y are completely random and independent of each other). E.g. from above if we take the unbiased die situation we get the following:

```
- 6*(1/6 * log(1/6)) - 6*(1/6 * log(1/6)) - - 6*6*(1/36 * log(1/36))
```

```
## [1] 0
```

I.e. when two die are perfectly unbiased we cannot improve our guess by rolling one of them first. Now if we take the biased die situation from above:

```
- 6*(1/6 * log(1/6)) - 6*(1/6 * log(1/6)) - - 6*(1/6 * log(1/6))
```

```
## [1] 1.791759
```

Here the mutual information is much higher, which means we should use knowledge of the joint distribution to inform us of a better guess.

The mutual information is entirely agnostic as to the nature of the relationship between variables (they can highly non-linear), and the concept can be extended to a large number of dimensions. This agnostic characteristic is both a blessing and a curse- it means we can apply the concept to complex data, but it does not reveal any information about the nature of the interaction present (e.g., positive or negative, or linear or non-linear correlation etc). This means in practice the mutual information is usually only a starting point for guiding a data miner towards variable dependencies that might be interesting to explore further. It was also an early form of multi-dimensional analysis statistic (dating back to at least the 1950s), since compared to modern expensive multi-dimensional model fitting (which are certainly more informative) it is fairly cheap to compute.

Correlation and Causation

A final thing to remember is that correlation does not necessarily mean causation. Users should pay heed to the lesson learned by Nicolas Cage, whose very appearance in films apparently drives people to an untimely drowning...

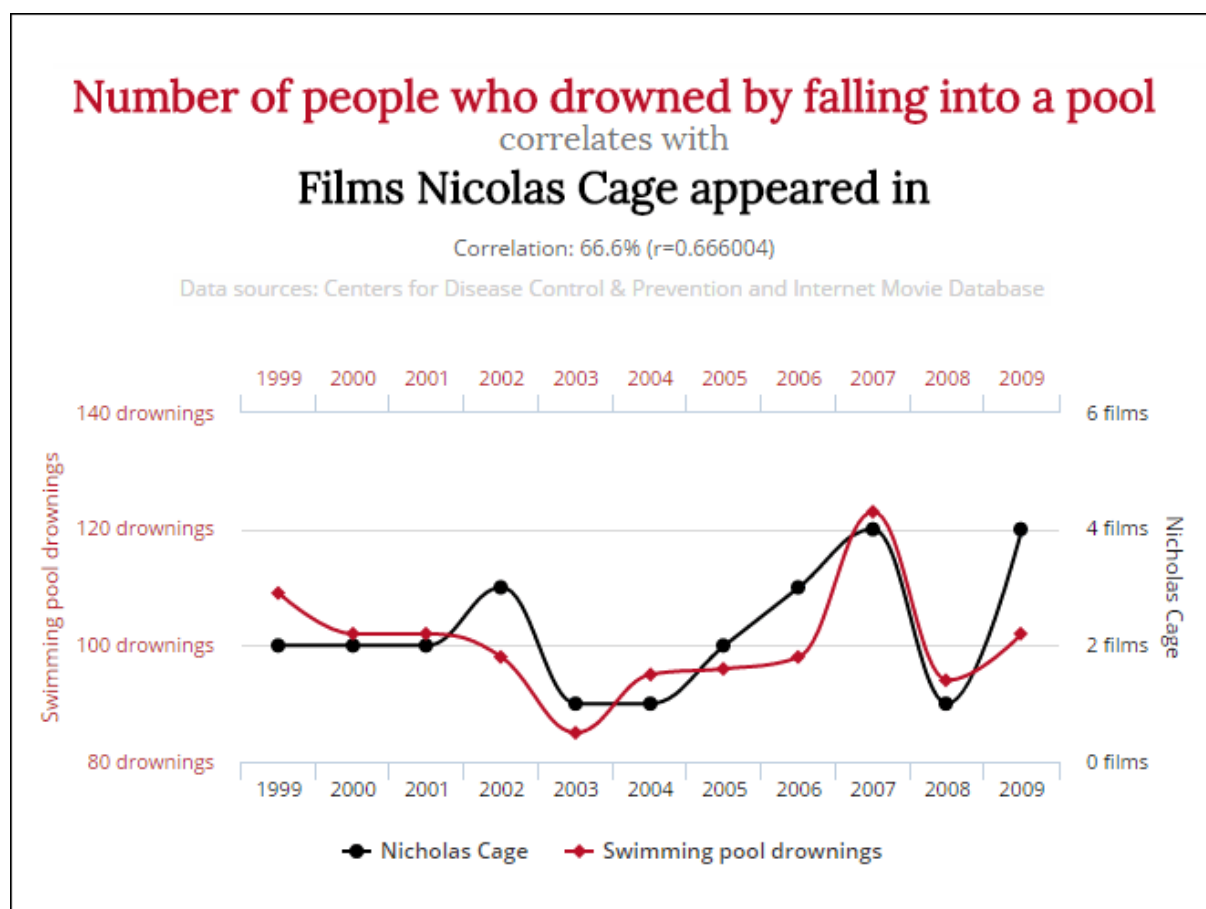


Figure 1: www.sadanduseless.com/crazy-stats/

One thing to note in the Cage correlation is that the number of data points from each distribution is not large (11). So we can test how likely such a correlation really is ($r^2 = 0.44$)

```
r2dist = foreach(i=1:1e3, .combine='c')%do%{cor(runif(11), runif(11))}
maghist(r2dist, xlab='Pearson Correlation', ylab='PDF', breaks=20)
```

```
## Summary of used sample:
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## -0.805359 -0.224542 -0.003694  0.003215  0.223501  0.891955
```

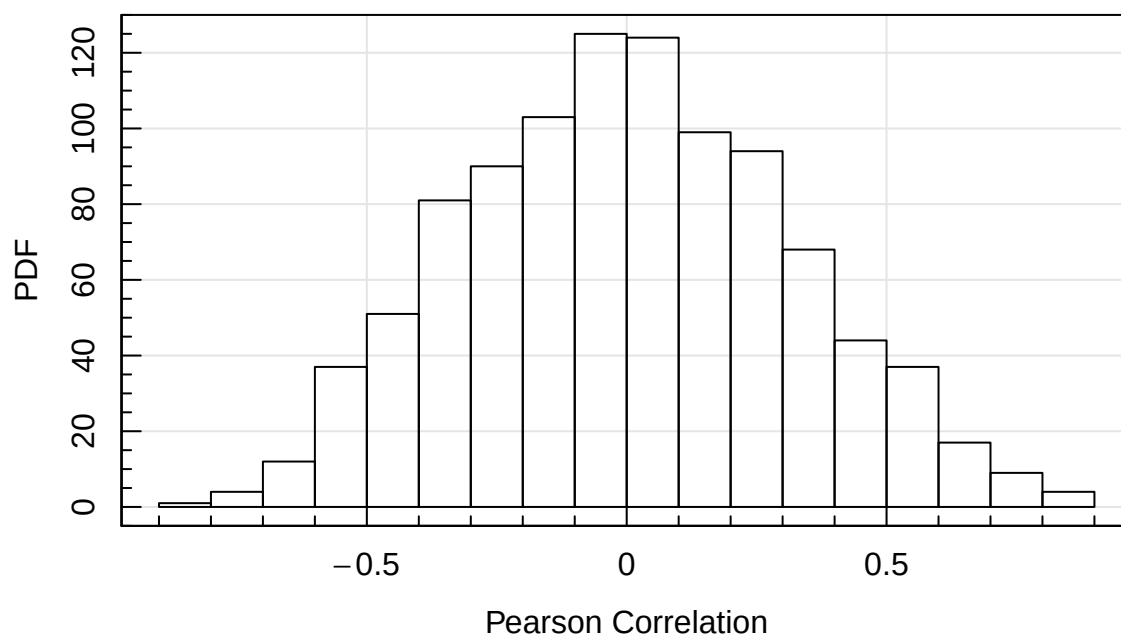
```
## Pop Std Dev: 0.31384
```

```
## MAD: 0.33066
```

```
## Half 16-84 Quan (1s): 0.32784
```

```
## Half 02-98 Quan (2s): 0.60253
```

```
## Using 1000 out of 1000
```

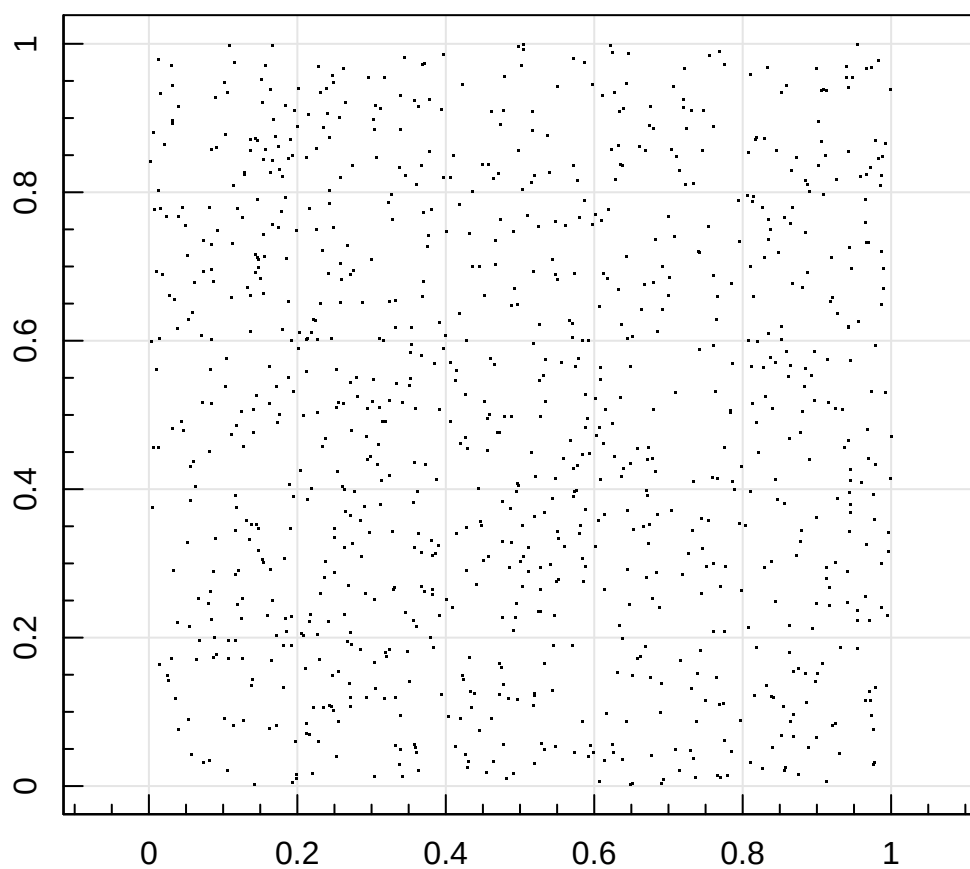
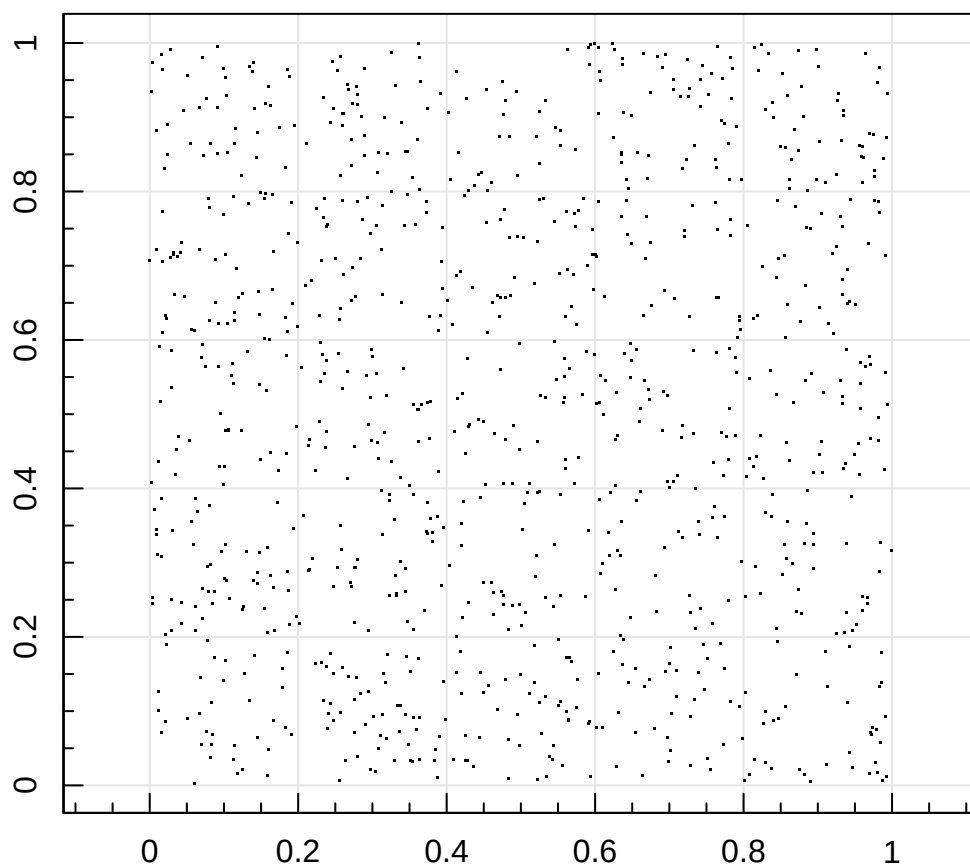


```
length(which(r2dist > 0.44))/1000
```

```
## [1] 0.091
```

From the above test we find ~9% of uncorrelated Uniform distributions will by chance have a Pearson $r^2 > 0.44$. So this means if we compare enough random things (roughly 10) then by chance we would get two fundamentally unrelated quantities that appear to correlate by chance. The above is a dangerous fact when data sets have a large number of measured quantities, but few observations- it becomes increasingly likely that two things with no inherent relationship with each appear to ‘significantly’ correlate.

We tend to be good at finding patterns in data too. Consider the 2 figures below, one of these has significantly more spatially correlated structure than the other. Which is which?



Whether or not we can reject the hypothesis that two quantities are truly unrelated is one of the many types of tests that can be achieved using Frequentist tests.

Fitting Linear Models

There is a common misconception that when undertaking multi-variate correlation analysis the choice of the method for the fitting and error handling is largely a matter of taste. However, if the problem is mathematically well-defined, the optimal fitting solution is, in most cases, unique. Here we look at the optimal fitting solution for the frequently encountered case schematised below, where the D -dimensional data ($D = 2$ in the figure) being analysed

- randomly samples a population characterised by a $(D - 1)$ -dimensional plane (a line if $D = 2$) with intrinsic Normal scatter,
- has multivariate Normal uncertainties (these may be heteroscedastic, i.e. different for each data point),
- has no defined predictor/response variable (i.e. a standard regression analysis would yield different solutions depending on axis-ordering).

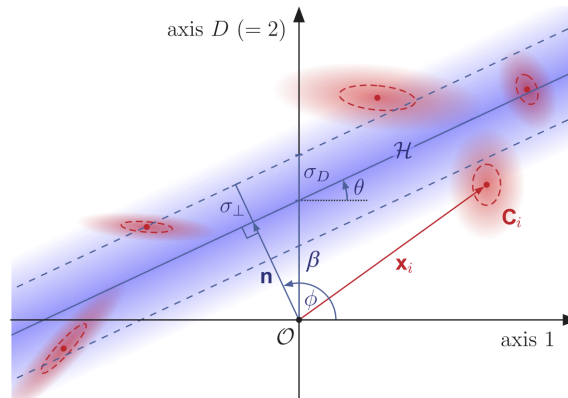


Figure 2: Linearly correlated data with heteroscedastic errors.

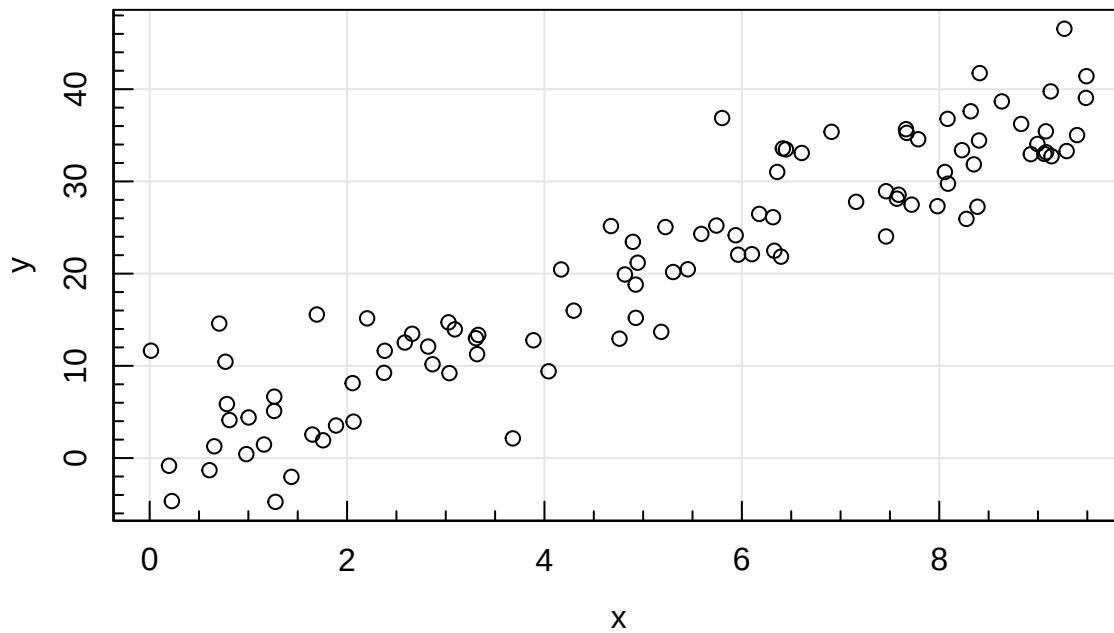
Upon assuming these properties the generative $(D - 1)$ -dimensional plane (hyperplane) that best explains the observed data is unique and can be fit using traditional likelihood analysis techniques. This is not a traditional regression analysis since we do not assume the data has a single response variable from one or more predictor variables. Instead we are considering the multi-dimensional generative model that best produces all the variables jointly. But first we will look at the common approaches to such regression problems.

The Bad, The Ugly and The Good

First let us make some simple data that would be linear except for constant observation errors that we know:

```
xdata = runif(1e2,0,10)
ydata = rnorm(1e2,mean=xdata*4,sd=5)

magplot(xdata,ydata,xlab='x',ylab='y')
```



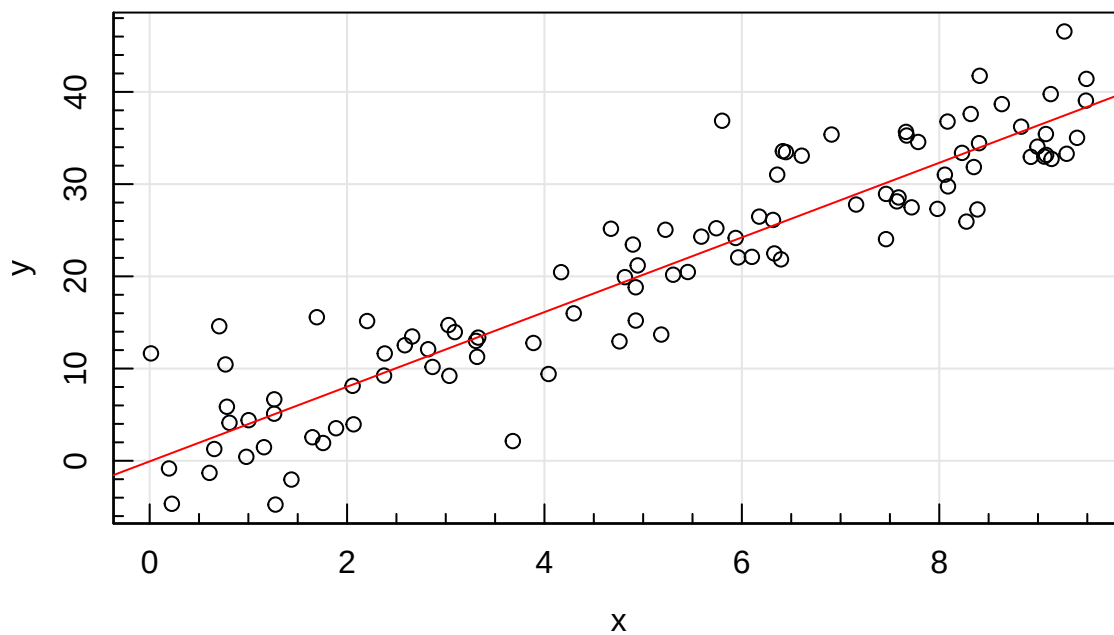
The Bad (Simple Regression)

As have seen previously, **R** has its own built in regression estimation function **lm** that works by minimising the variance of the fit residual along a response axis. We can use this to estimate a simple regression fit to the above:

```
lm_out1 = lm(ydata~xdata)
lm_out1$coefficients # The y=a.x+b regression terms (offset b is [1], gradient a is [2])

## (Intercept)      xdata
## -0.06708167  4.04734840

magplot(xdata, ydata, xlab='x', ylab='y')
abline(lm_out1, col='red')
```



But what happens if we are actually agnostic about the response and predictor axes? This would be akin to doing regression of x instead (i.e. minimising variance in these residuals instead):

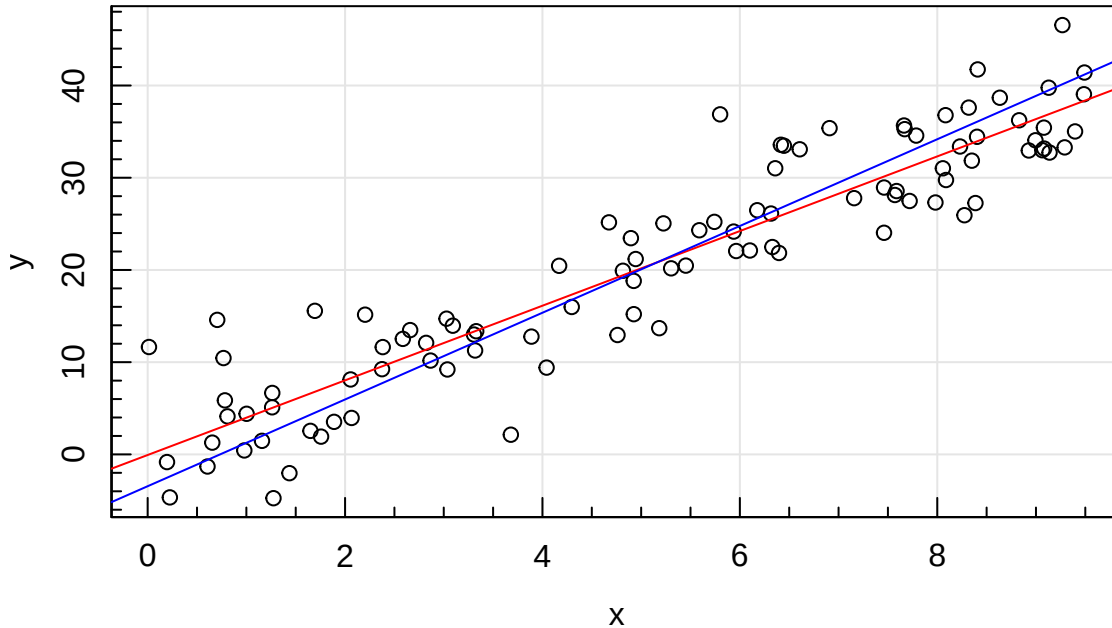
```
lm_out2 = lm(xdata~ydata)
```

Even though we have made the fit using x regression, we can still describe it in the usual $y = \alpha x + \beta$ from:

```
xreg_b = -lm_out2$coefficients[1]/lm_out2$coefficients[2]
xreg_a = 1/lm_out2$coefficients[2]
```

And we can over-plot the solution:

```
magplot(xdata, ydata, xlab='x', ylab='y')
abline(lm_out1, col='red')
abline(xreg_b, xreg_a, col='blue')
```

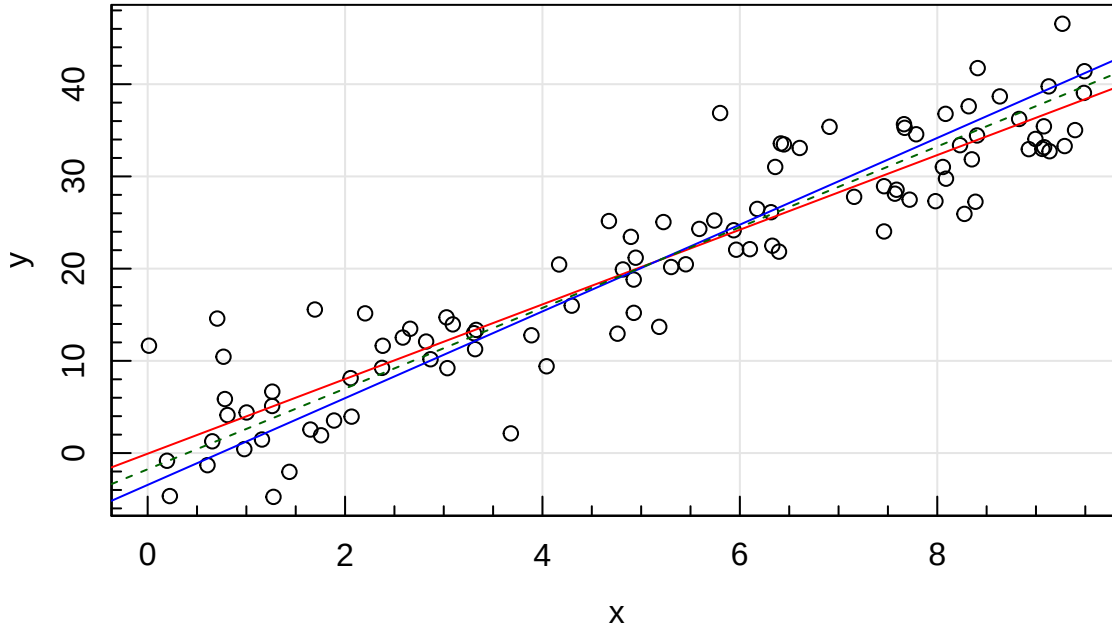


So disconcertingly, the order in which we do the regression certainly has an impact on the result. In astronomy it is not unusual to have no predictor/response dichotomy (e.g. colour-magnitude and mass-size plots) since we tend to measure all values rather than setting any (as we might in a physics experiment say, where the split makes more sense). So why would we want to state a different physical insight just on a whim of column ordering? We clearly do not, so what we need is a rotation invariant regression system.

The Ugly (Bisector Method)

As a work around, for many years astronomers made use of the bisector and/or averaging method. As the name implies this method aims to create a regression that is between the two solutions. E.g.:

```
magplot(xdata, ydata, xlab='x', ylab='y')
abline(lm_out1, col='red')
abline(xreg_b, xreg_a, col='blue')
abline(mean(c(lm_out1$coefficients[1],xreg_b)), mean(c(lm_out1$coefficients[2],xreg_a)),
        col='darkgreen', lty=2)
```



This works somewhat visually, but it is still not robust to arbitrary rotations of the data since neither the x and y regression are robust to this, i.e. the angle of the gradient change in the regression can be different to the angle of the data rotation (prove this yourself with some examples). Also, this approach will not scale to higher dimensions, since there is no guarantee that a unique bisector hyperplane exists for 3 or more hyperplanes.

The Good (Full Bayesian Approach)

Luckily this problem can be tackled in a rigorous Bayesian manner, which gets us to the rotation invariant domain we desire, and can even cope with intrinsic scatter and variable data error with covariance. First we will develop the general theory for fitting data points in D dimensions by a $(D - 1)$ -dimensional plane, such as fitting points in two dimensions by a line (this is largely taken from Robotham & Obreschkow, 2015 - RO15 from here).

We assume that the data represents a random sample of a population exactly described by a $(D - 1)$ -dimensional plane with intrinsic Normal scatter, referred to as the generative model. The objective is to determine the most likely generative model, by fitting simultaneously for the $(D - 1)$ -dimensional plane and the intrinsic Normal scatter, given data points with multivariate Normal uncertainties : the uncertainties are (1) different for each data point (heteroscedasticity), (2) independent between different data points (Markovianity), and (3) covariant between orthogonal directions, e.g. $\sim x$ -errors and y -errors can be correlated.

Consider N data points $i = 1, \dots, N$ in D dimensions. These points are specified by the position vectors x_i relative to a fixed origin (O) and Normal uncertainties expressed by the covariance matrices C_i . Thus, the probability distribution function (PDF) that fully describes our knowledge about a point i is

$$\rho_i(x) = \frac{1}{\sqrt{(2\pi)^D |C_i|}} e^{-\frac{1}{2}(x-x_i)^\top C_i^{-1}(x-x_i)}.$$

These data are assumed to derive from a generative model given by a hyperplane (H , a line if $D = 2$, a plane if $D = 3$) with Normal scatter $\sigma_\perp$ orthogonal to H . Note that the scatter then remains Normal along any other direction, not parallel to H . The hyperplane is parameterised by the normal vector $n \perp H$ going from O to H . Hence, the distance of a point x from H equals $|\hat{n}^\top x - n|$, where $\hat{n} = n/|n|$. Thus, the hyperplane is the ensemble of points that satisfy

$$\hat{n}^\top x - n = 0$$

Given this parametrisation, the generative model is fully described by the PDF

$$\rho_m(x) = \frac{1}{\sqrt{2\pi\sigma_\perp^2}} e^{-\frac{(\hat{n}^\top x - n)^2}{2\sigma_\perp^2}}.$$

The likelihood of a data point i then equals

$$L_i = \int dx \rho_i(x) \rho_m(x) = \frac{1}{\sqrt{2\pi s_{\perp,i}^2}} e^{-\frac{(\hat{n}^\top x_i - n)^2}{2s_{\perp,i}^2}},$$

where $s_{\perp,i}^2 \equiv \sigma_\perp^2 + \hat{n}^\top C_i \hat{n}$. This can be thought of as summing in quadrature the covariant error in the direction orthogonal to the hyperplane with the intrinsic scatter of the plane. Following Bayesian theory and assuming default Uniform priors on $\hat{n}$ and $\sigma_\perp$, the most likely model is obtained by maximising the total likelihood $L = \prod_{i=1}^N L_i$ or its logarithm $\ln L$. Up to an additive constant $\frac{N}{2} \ln(2\pi)$,

$$\ln L = -\frac{1}{2} \sum_{i=1}^N \left[\ln(s_{\perp,i}^2) + \frac{(\hat{n}^\top x_i - n)^2}{s_{\perp,i}^2} \right].$$

Whilst writing down the likelihood is usually considered to be the hard part in any Bayesian problem, actually encoding it carefully can be challenging too. In the above there are many coordinate transformations to worry about, and it is deliberately written in a manner that is dimension agnostic, i.e. in principle it should work on 10 dimensional hyperplanes as well as simple 2D line fits.

For this reason writing, documenting and supporting computer implementations of complex statistical tools is an important factor in modern day research, since it is too onerous to expect a young researcher to re-implement all modern analysis techniques from scratch *and* conduct interesting research. To that end we provide **hyper.fit** as a maintained package on CRAN that is routinely used to do science (mainly by astronomers as far as I can tell). To further encourage the uptake of a better approach to fitting a line, I also wrote a Shiny website that abstracts away the need to know any **R** (hyperfit.icrar.org/).

The Hyper Fit Package

With the philosophy of doing rigorous Bayesian regression with proper error accounting, as part of the paper of RO15 we released the **R** package **hyper.fit**. This is available from CRAN, and can be loaded in the usual way:

```
library(hyper.fit)
```

```
## Loading required package: rgl
```

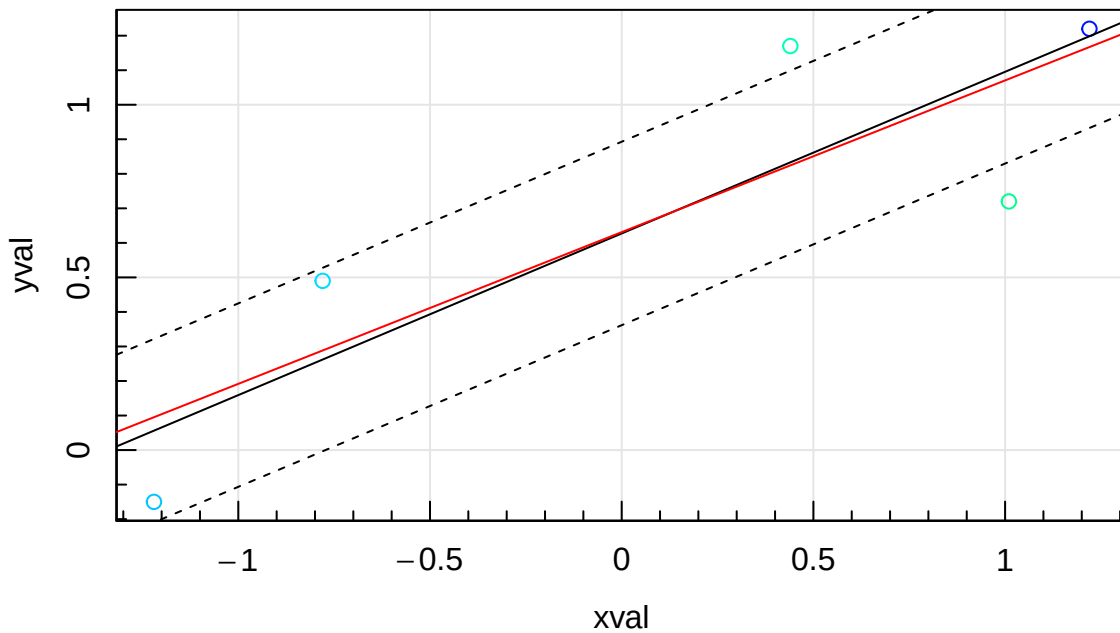
```
## Loading required package: LaplacesDemon
```

To get going we will define the data that went into the schematic scatter plot above:

```
xval = c(-1.22, -0.78, 0.44, 1.01, 1.22)
yval = c(-0.15, 0.49, 1.17, 0.72, 1.22)
```

Due to a number of internal methods to guess good starting points, we can fit and plot this data fairly immediately, and without the usual complexity of numerically describing the above likelihood model or priors etc. This is handy since the above is pretty laborious to encode (and prone to user error).

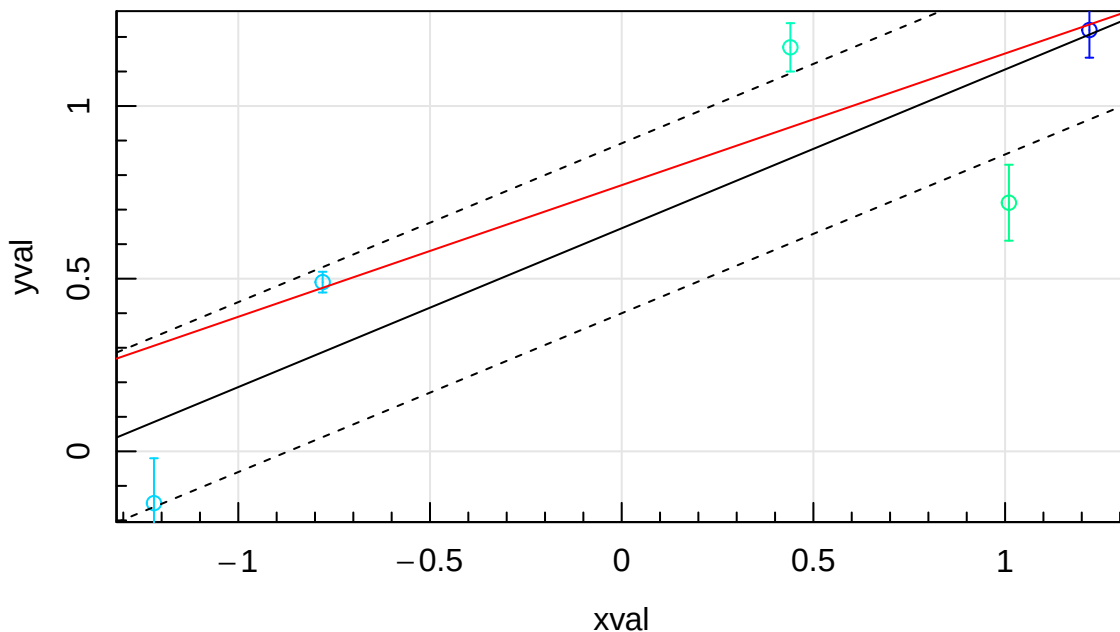
```
fitnoerror = hyper.fit(cbind(xval, yval))
plot(fitnoerror) #hyper.fit incorporates class based plotting
abline(lm(yval ~ xval), col='red')
```



The red line is what we would achieve using the standard regression approach (`lm`, discussed in an earlier chapter) that is build into **R**. However, this method is not Bayesian, and it does not allow for proper error consideration. It does allow for *weights* for the observations, but this is an ad hoc route at best.

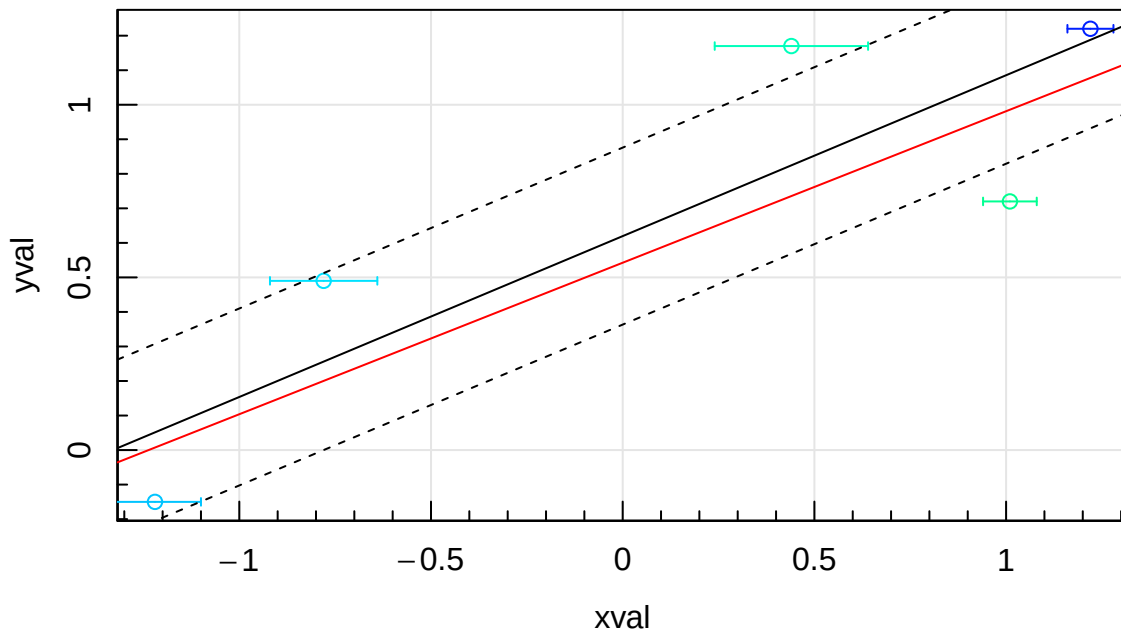
At this stage we are ignoring errors, so the next step is to add these. First just y axis error:

```
yerr = c(0.13, 0.03, 0.07, 0.11, 0.08)
fitwithyerror = hyper.fit(cbind(xval, yval), vars=cbind(0, yerr)^2)
plot(fitwithyerror)
abline(lm(yval ~ xval, weights=1/yerr^2), col='red')
```



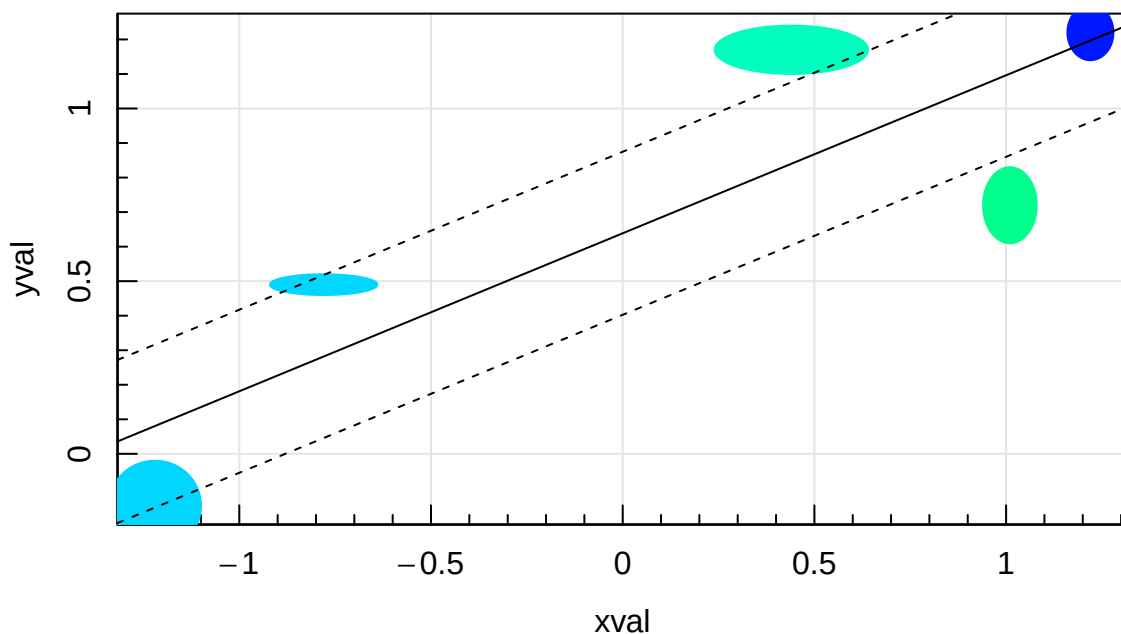
The above achieves a similar fit but with y errors now also being used (and plotted). The use of *weights* in `lm` clearly changes things a lot! Next we use x errors:

```
xerr = c(0.12, 0.14, 0.20, 0.07, 0.06)
fitwithxerror = hyper.fit(cbind(xval, yval), vars=cbind(xerr, 0)^2)
plot(fitwithxerror)
abline(lm(yval ~ xval, weights=1/xerr^2), col='red')
```



Again, a similar fit is achieved when using the x errors, and the **lm** fit looks quite different. Now we can use both x and y errors together (but with no covariance):

```
fitwithxyerror = hyper.fit(cbind(xval, yval), vars=cbind(xerr, yerr)^2)
plot(fitwithxyerror)
```



We now see ellipses for the errors. The final part of the Hyper-Fit method we outlined above was the option of including covariance (or in this case, correlation) in the errors. In the background this requires the propagation of an array that is $2 \times 2 \times 5$, where each layer is the per observation covariance matrix. Handily **hyper.fit** includes a helper function to do this easily.

```
xycor = c(0.90, -0.40, -0.25, 0.00, -0.20)
covarray = makecovarray2d(xerr, yerr, xycor)
covarray
```

```
## , , 1
##
##      [,1] [,2]
## [1,] 0.01440 0.01404
```

```
## [2,] 0.01404 0.01690
##
## , , 2
##
##      [,1]      [,2]
## [1,] 0.01960 -0.00168
## [2,] -0.00168 0.00090
##
## , , 3
##
##      [,1]      [,2]
## [1,] 0.0400 -0.0035
## [2,] -0.0035 0.0049
##
## , , 4
##
##      [,1]      [,2]
## [1,] 0.0049 0.0000
## [2,] 0.0000 0.0121
##
## , , 5
##
##      [,1]      [,2]
## [1,] 0.00360 -0.00096
## [2,] -0.00096 0.00640
```

```
covarray[, , 1]
```

```
##      [,1]      [,2]
## [1,] 0.01440 0.01404
## [2,] 0.01404 0.01690
```

```
xerr[1]^2
```

```
## [1] 0.0144
```

```
yerr[1]^2
```

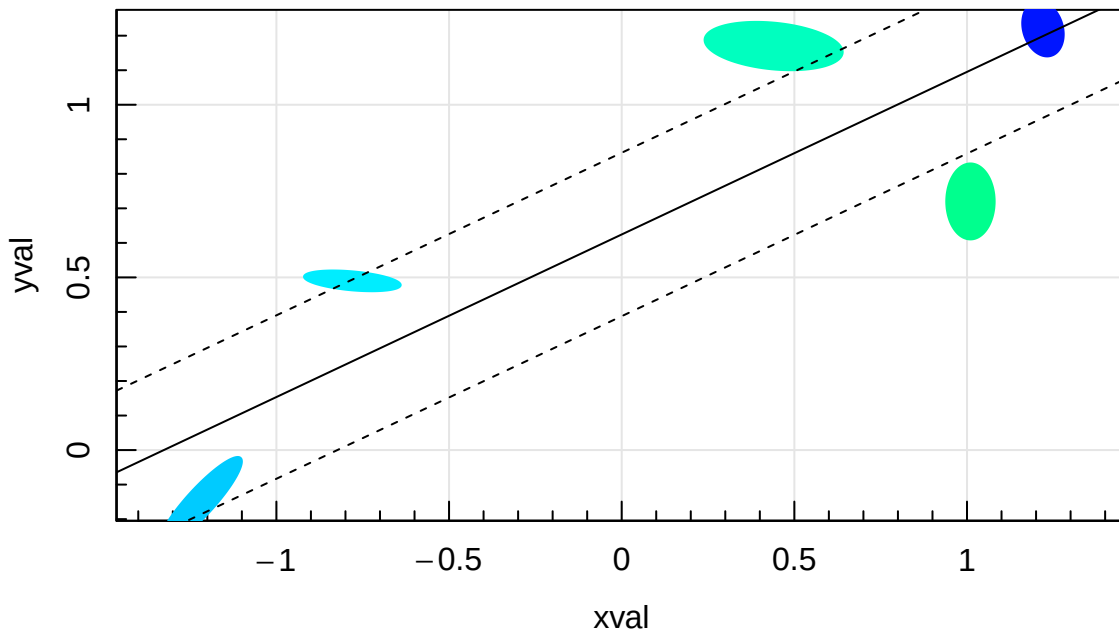
```
## [1] 0.0169
```

```
xerr[1]*yerr[1]*xycor[1]
```

```
## [1] 0.01404
```

We pass this array directly into *hyper.fit*:

```
fitwitherrorandcor = hyper.fit(cbind(xval, yval), covarray=covarray)
plot(fitwitherrorandcor, asp=1)
```



Now we can see the ellipses have changed shape (the semi-major axis length will be the same) and some are rotated. Again, the actual fit has not radically changed, but we are now describing the process in a rigorous Bayesian manner.

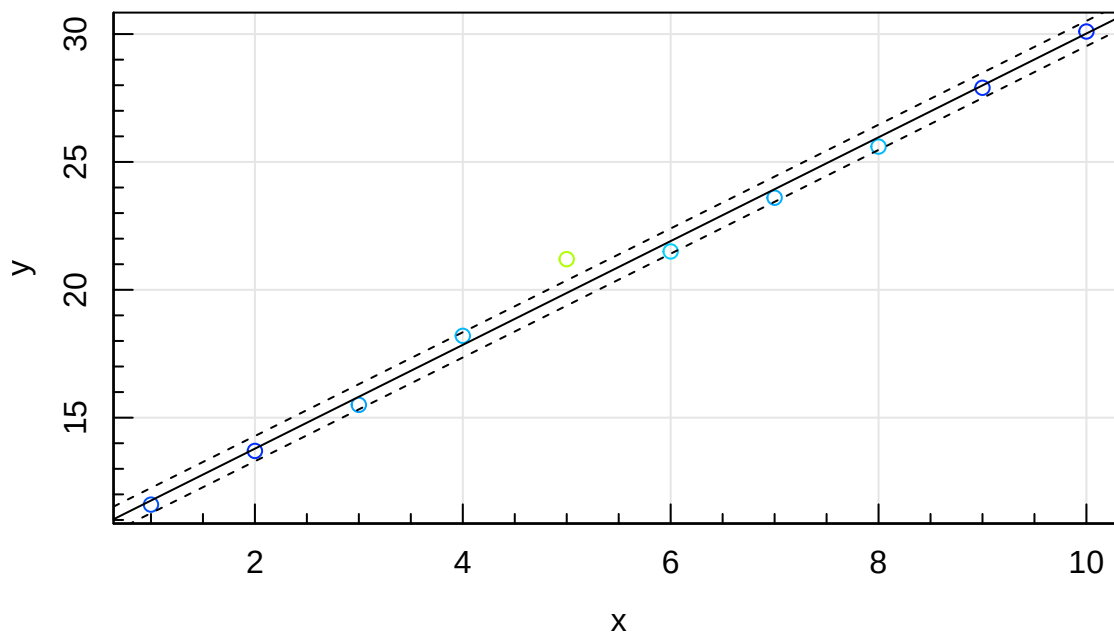
Different Parameterisations

Hyper-Fit offers a number of coordinate schemes to define the hyperplane:

```
simple = cbind(x=1:10,y=c(11.6, 13.7, 15.5, 18.2, 21.2, 21.5, 23.6, 25.6, 27.9, 30.1))
fitsimple = hyper.fit(simple)
summary(fitsimple)
```

```
## Call used was:
##
## hyper.fit(X = simple)
##
## Data supplied was 10rows x 2cols.
##
## Requested parameters:
##
##      alpha1 beta.vert scat.vert
## 2.0292690 9.7289884 0.4934046
##
## Errors for parameters:
##
##      err_alpha1 err_beta.vert err_scat.vert
##      0.05447899   0.33782604   0.11082873
##
## The full parameter covariance matrix:
##
##              alpha1    beta.vert    scat.vert
## alpha1      0.0029679600 -0.016323855  0.0005801598
## beta.vert   -0.0163238554  0.114126433 -0.0031924720
## scat.vert   0.0005801598 -0.003192472  0.0122830076
##
## The sum of the log-likelihood for the best fit parameters:
##
```

```
##      LL
## 10.22736
##
## Unbiased population estimator for the intrinsic scatter:
##
## scat.vert
## 0.5691087
##
## Standardised parameters vertically projected along dimension 2 (y):
##
##      alpha1 beta.vert scat.vert
## 2.0292690 9.7289884 0.4934046
##
## Standardised generative hyperplane equation with unbiased population
## estimator for the intrinsic scatter, vertically projected along dimension 2 (y):
##
## y ~ N(mu= 2.029*x + 9.729 , sigma= 0.5691)
plot(fitsimple)
```



The three plane systems are of type $y = \alpha x + \beta$, $y = \tan(\theta)x + \beta$, and the normal vector from the hyperplane to the origin (n) defined by x and y components n_1 and n_2 . Separately to how we define the hyperplane itself is how to parameterise the intrinsic scatter around it. Standard convention is to describe it as the scatter orthogonal to the plane, or along the ‘vertical’ axis, which is usually the last (so y in 2D and z in 3D). By definition, it is at its smallest when defined to be orthogonal to the hyperplane itself (which we can see above).

Depending on the data being used, you might find better behaved posteriors (more Normal) are achieved using different hyperplane coordinate definition schemes:

```
print(hyper.fit(simple, coord.type='alpha', scat.type='orth')$parm)
```

```
##      alpha1 beta.vert scat.orth
## 2.0292664 9.7291031 0.2182026
```

```
print(hyper.fit(simple, coord.type='theta', scat.type='orth')$parm)
```

```
##      theta1 beta.vert scat.orth
## 63.7666239 9.7288669 0.2181231
```

```

print(hyper.fit(simple, coord.type='normvec', scat.type='orth')$parm)

##  normvec1  normvec2  scat.orth
## -3.8575755  1.9009638  0.2181034

print(hyper.fit(simple, coord.type='alpha', scat.type='vert.axis')$parm)

##  alpha1 beta.vert scat.vert
## 2.0292690 9.7289884 0.4934046

print(hyper.fit(simple, coord.type='theta', scat.type='vert.axis')$parm)

##  theta1 beta.vert scat.vert
## 63.7666860 9.7289188 0.4934103

print(hyper.fit(simple, coord.type='normvec', scat.type='vert.axis')$parm)

##  normvec1  normvec2  scat.vert
## -3.8575135  1.9009136  0.4934627

```

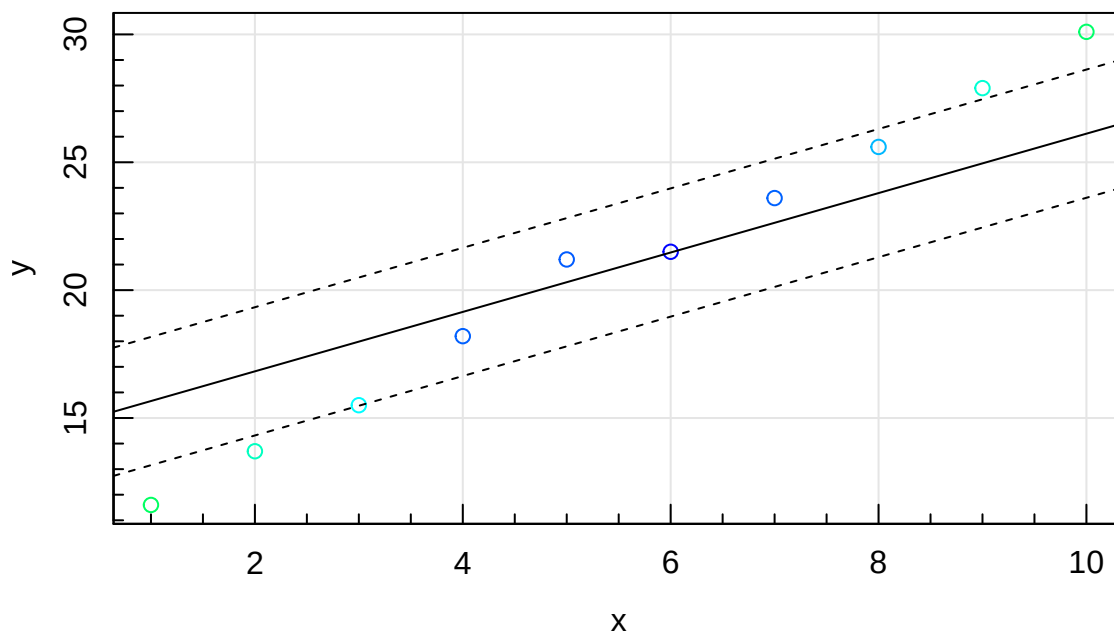
Using Priors

So far we have assumed priors that are effectively Uniform (and improper) on angle, thus making our Hyper-Fit approach rotation agnostic, with no defined predictor and/or response axes. Being Bayesian minded, it is of course possible to add priors into Hyper-Fit, and in fact these serve as an elegant way to *fix* parameter values to a required quantity, should the need arise. The *prior* option takes a parameter as the input which is passed the parameters, and whatever is then done using these has to produce a log-likelihood that is added to the data-model *LL* to produce our *LP* for optimisation. The default just sets the result of *prior* to 0.

```

fitsimple_prior = hyper.fit(simple, prior=function(parm){dnorm(parm[1],mean=1,sd=0.1,log=TRUE)})
plot(fitsimple_prior)

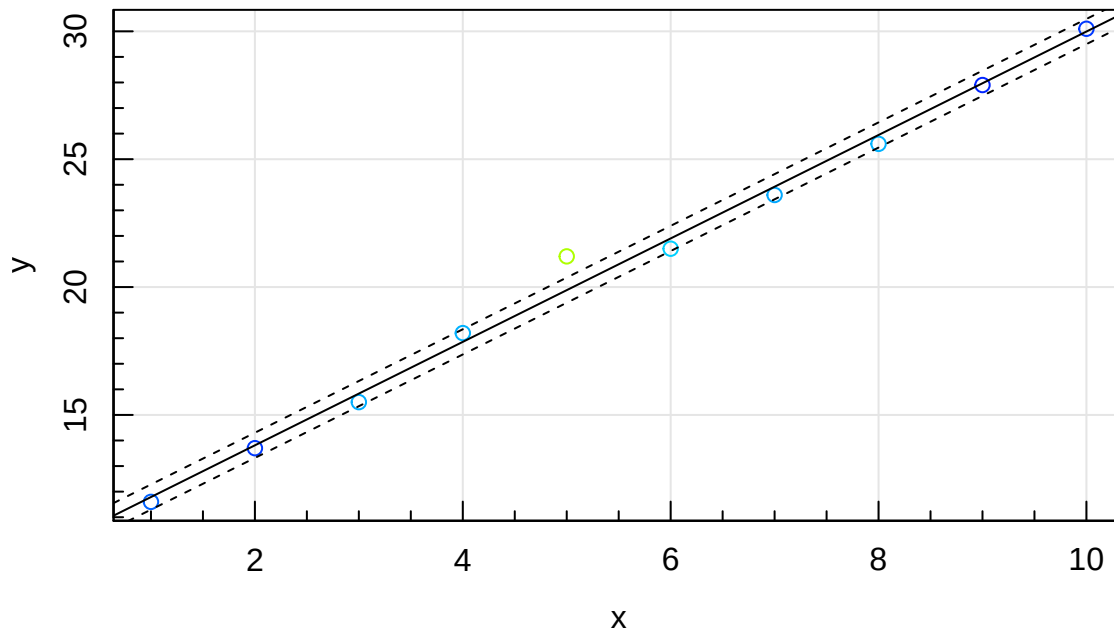
```



```

fitsimple_prior = hyper.fit(simple, prior=function(parm){dnorm(parm[1],mean=2,sd=0.1,log=TRUE)})
plot(fitsimple_prior)

```



The first case shows the impact of using a poor prior, where the resulting α is dragged a long way towards the value of 1, rather than coming out as 2 (the correct answer). The second case uses a reasonable prior close to the right value, and this behaves much like the earlier fit.

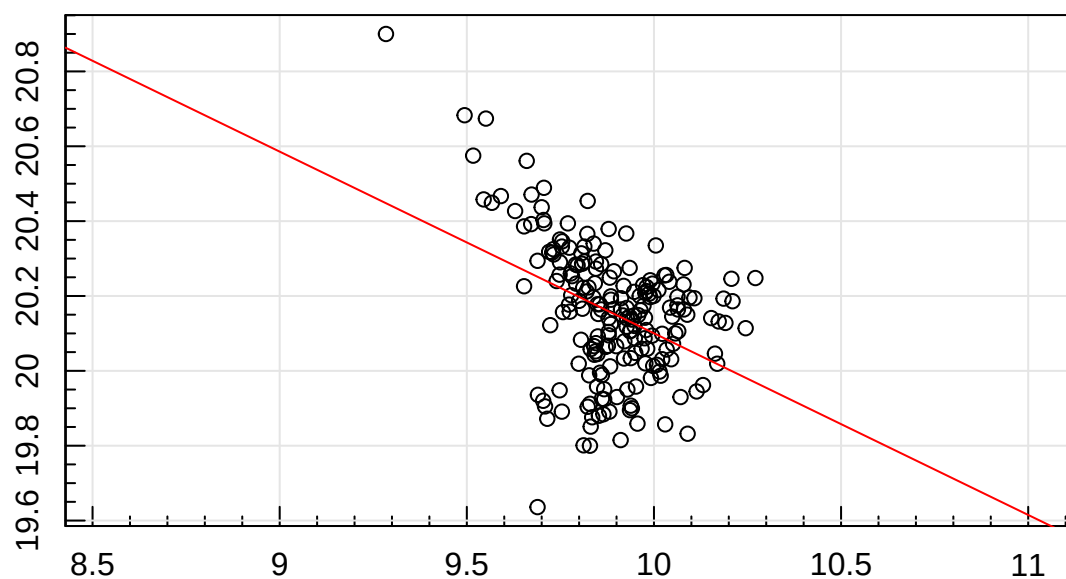
Tricky Cases

To make the point about why a rotation agnostic Bayesian line/plane/hyperplane fitting method is best, the package comes with some difficult and interesting test cases.

Trumpet Data

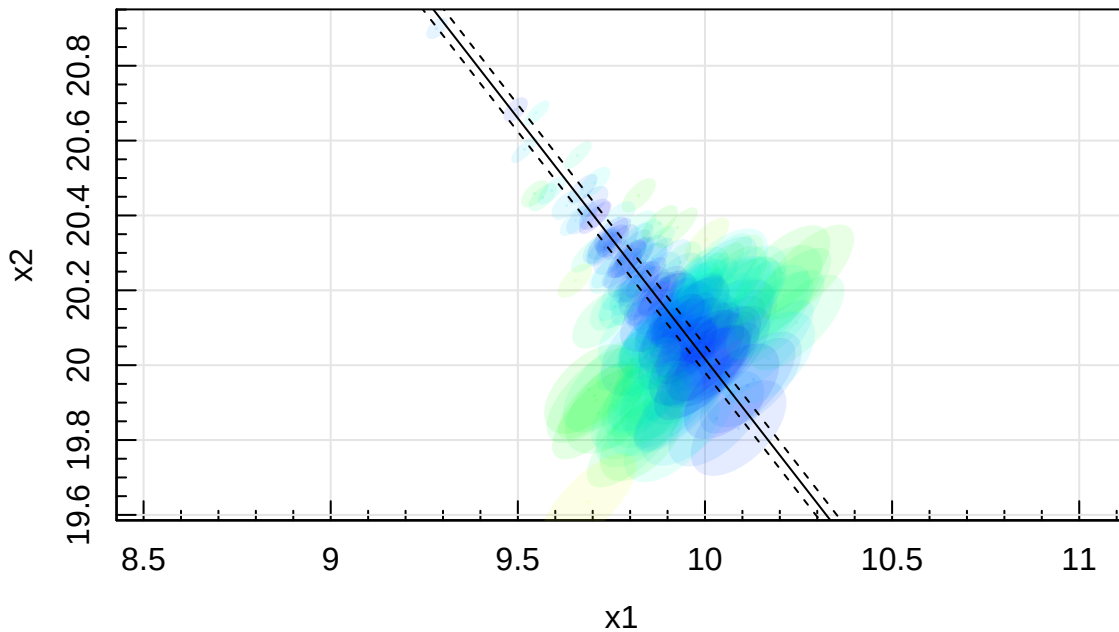
Consider the trumpet test data:

```
data(trumpet)
magplot(trumpet[,1:2],asp=1)
abline(lm(trumpet[,2] ~ trumpet[,1]),col='red')
```



```
fittrumpet = hyper.fit(X=cbind(trumpet$x, trumpet$y), covarray=makecovarray2d(trumpet$x_err,
trumpet$y_err, trumpet$corxy), coord.type='normvec')
```

```
plot(fittrumpet, trans=0.1, pch='.', asp=1)
```



Clearly knowledge of the error covariance is key in correctly orienting the line through the data, and when this is achieved it is clear there is actually very little scatter beyond that due to the errors, which happen to grow as we move down the line.

Fundamental Plane with 6dFGS

The package also comes loaded with public 6dFGS fundamental plane data, which we load much like before:

```
data(FP6dFGS)
```

For this data set we can also make use of *weights*, that represent how incomplete different regions of parameter space are expected to be, not the error on the observed measurement (which is still defined as before):

```
fitFP6dFGSw = hyper.fit(FP6dFGS[,c('logIe_J', 'logsigma', 'logRe_J')],
vars = FP6dFGS[,c('logIe_J_err', 'logsigma_err', 'logRe_J_err')]^2, weights=FP6dFGS[, 'weights'],
coord.type='alpha', scat.type='vert.axis')
summary(fitFP6dFGSw)
```

```
## Call used was:
##
## hyper.fit(X = FP6dFGS[, c("logIe_J", "logsigma", "logRe_J")],
##   vars = FP6dFGS[, c("logIe_J_err", "logsigma_err", "logRe_J_err")]^2,
##   weights = FP6dFGS[, "weights"], coord.type = "alpha", scat.type = "vert.axis")
##
## Data supplied was 8803rows x 3cols.
##
## Requested parameters:
##
##      alpha1      alpha2    beta.vert    scat.vert
## -0.85256972  1.50822289 -0.42050696  0.05989333
##
## Errors for parameters:
##
##      err_alpha1    err_alpha2 err_beta.vert err_scat.vert
##      0.004680542    0.012787689  0.031151680  0.001448929
##
```

```
## The full parameter covariance matrix:
##
##          alpha1      alpha2      beta.vert      scat.vert
## alpha1      2.190747e-05 -5.945010e-06 -5.689288e-05  1.016979e-07
## alpha2     -5.945010e-06  1.635250e-04 -3.492476e-04  5.485396e-06
## beta.vert   -5.689288e-05 -3.492476e-04  9.704272e-04 -1.271260e-05
## scat.vert   1.016979e-07  5.485396e-06 -1.271260e-05  2.099395e-06
##
## The sum of the log-likelihood for the best fit parameters:
##
##      LL
## 28775.4
##
## Unbiased population estimator for the intrinsic scatter:
##
##      scat.vert
## 0.05990354
##
## Standardised parameters vertically projected along dimension 3 (logRe_J):
##
##      alpha1      alpha2      beta.vert      scat.vert
## -0.85256972  1.50822289 -0.42050696  0.05989333
##
## Standardised generative hyperplane equation with unbiased population
## estimator for the intrinsic scatter, vertically projected along dimension 3 (logRe_J):
##
## logRe_J ~ N(mu= -0.8526*logIe_J + 1.5082*logsigma - 0.4205 , sigma= 0.0599)
```

We can view this fit in 3D interactively in **R** by running

```
plot(fitFP6dFGSw)
```

Below are a few static shots of the 3D view:

In comparison **lm** does a hopelessly poor job of describing the true plane:

```
lm(FP6dFGS[, "logRe_J"] ~ FP6dFGS[, "logIe_J"] + FP6dFGS[, "logsigma"])
```

```
##
## Call:
## lm(formula = FP6dFGS[, "logRe_J"] ~ FP6dFGS[, "logIe_J"] + FP6dFGS[,
##      "logsigma"])
##
## Coefficients:
##      (Intercept)      FP6dFGS[, "logIe_J"]      FP6dFGS[, "logsigma"]
##           0.8708                -0.7226                0.7610
```

Principal Component Analysis

Principal Component Analysis (PCA) extends the concept of simple 2D regression to higher dimensions. Considering the simple linear regression model, PCA effectively attempts to fit a linear model, subtract it, and fit a new orthogonal model until all of the variance in the data is explained (remember that is what a standard regression achieves- the linear model that explains most of the variance in a certain direction). The idea is that if you can explain a significant fraction of the data variance (say 90%) with fewer components than parameters present (say 3 components for a 6 parameter data set) then you have efficiently reduced the useful dimensionality of the data.

As a simple example consider this data:

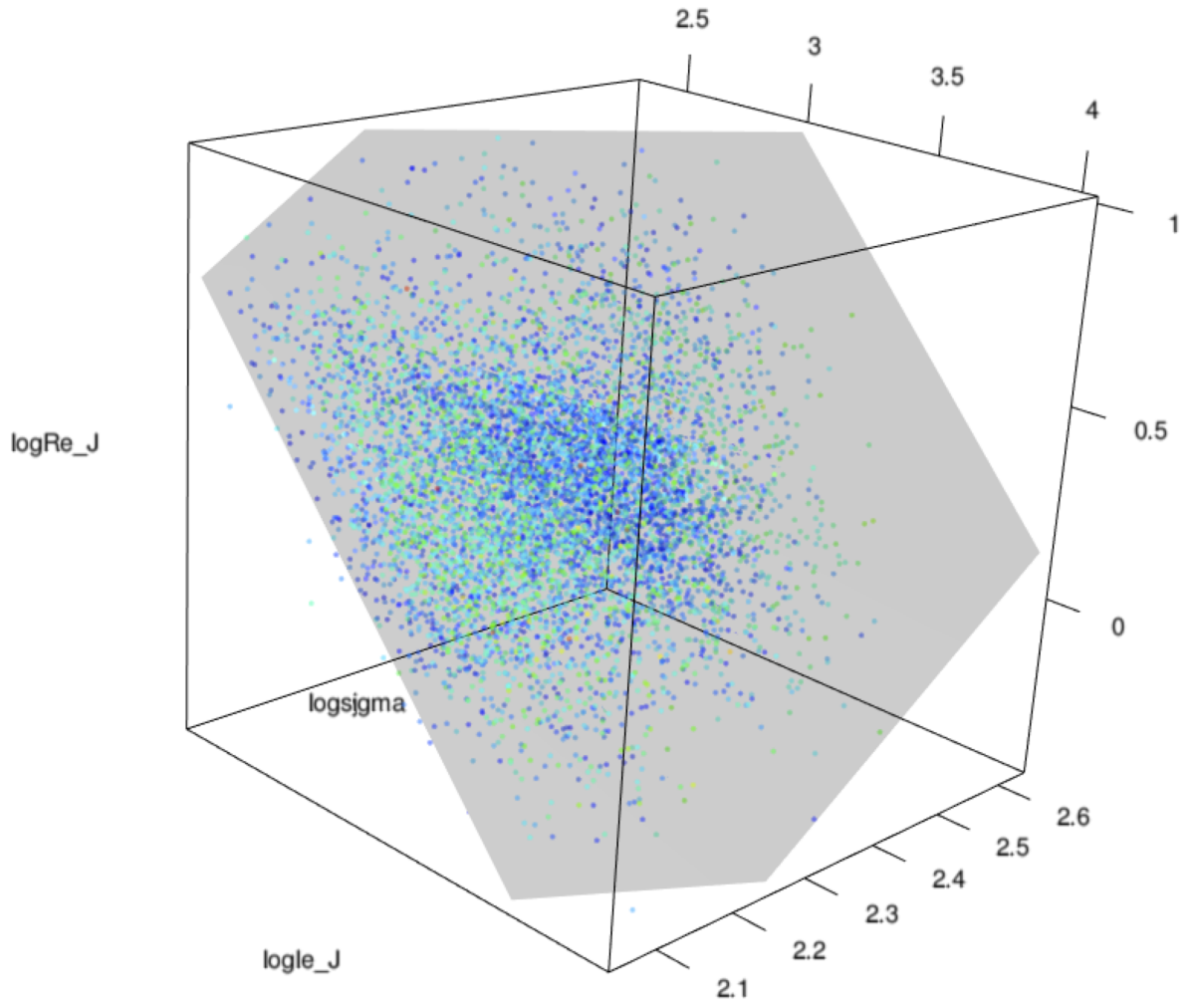


Figure 3: Angled view of 6dFGS fundamental plane.

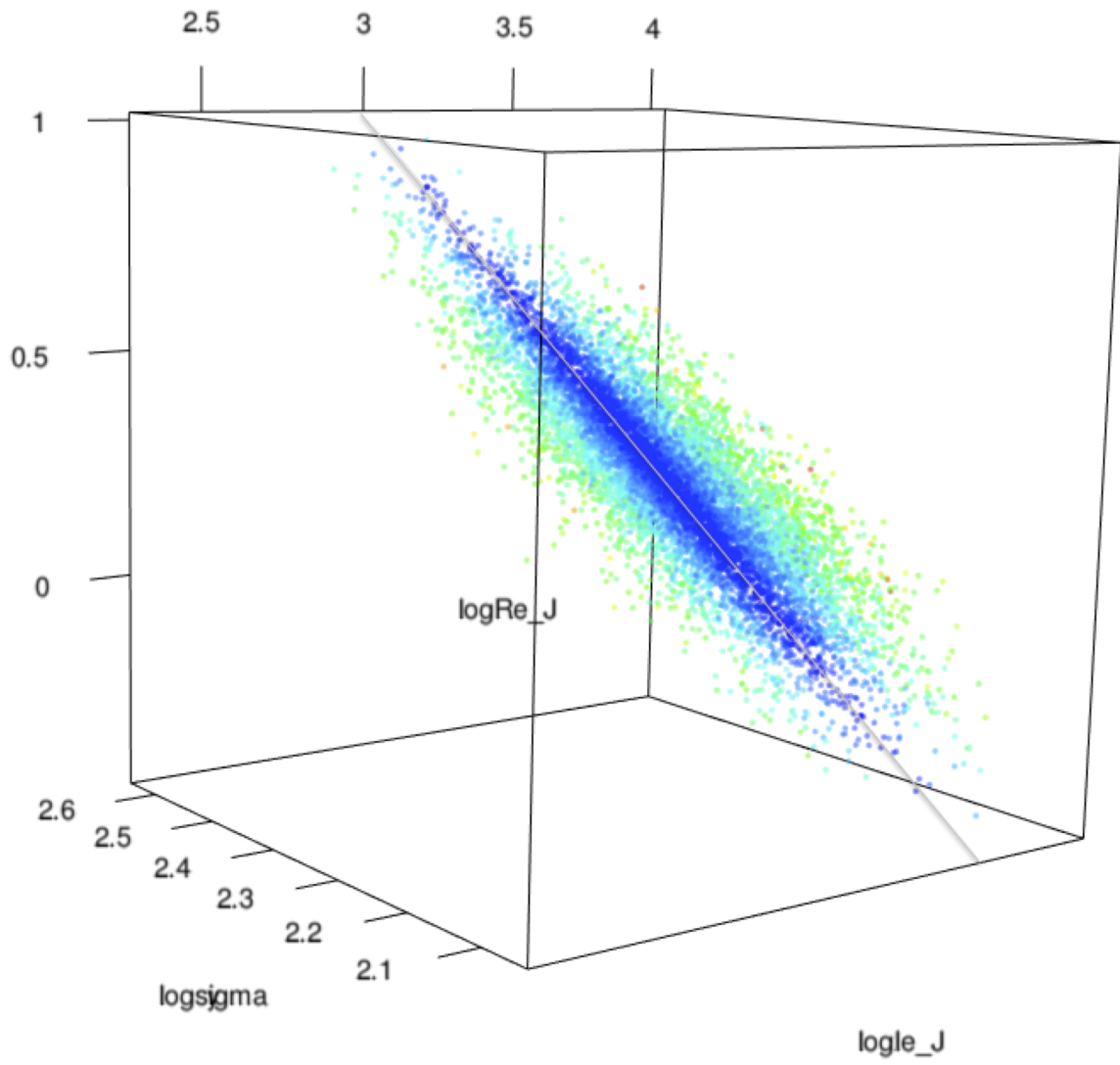
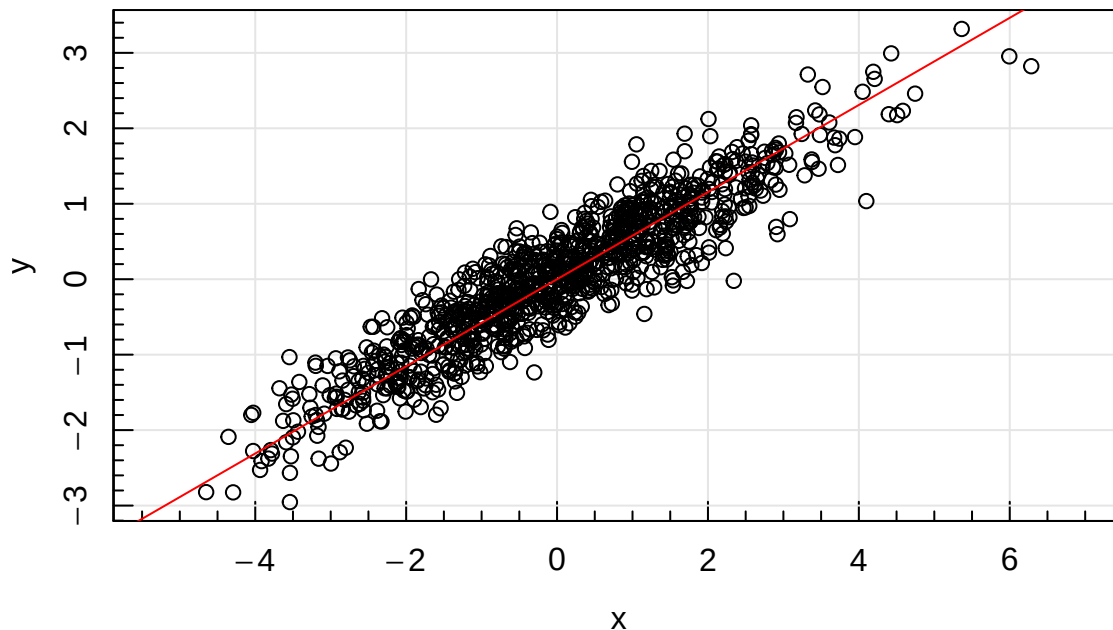


Figure 4: Edge on view of 6dFGS fundamental plane.

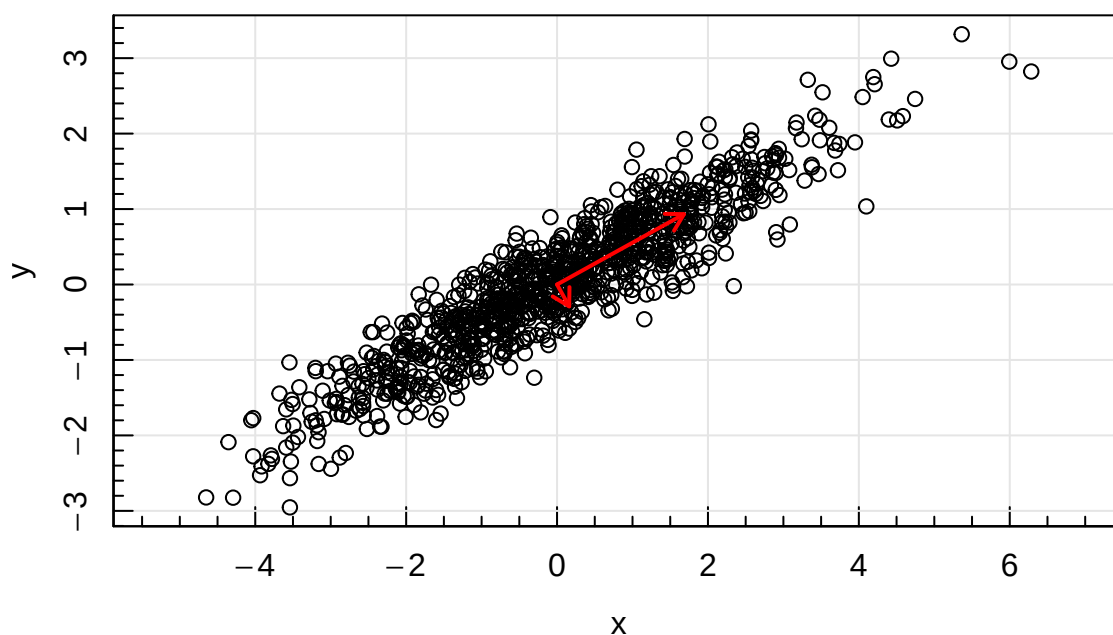
```
PCAmock = mvrnorm(1e3,mu=c(0,0),Sigma=matrix(c(3,1.6,1.7,1),2))
magplot(PCAmock, asp=1, xlab='x', ylab='y')
abline(0,1/sqrt(3),col='red')
```



The first component containing most of the variance will be the vector that follows the usual linear regression line (the red line here). Clearly collapsing the data down along this vector removes *most* of the variance we see, but not all. A second component the is orthogonal to the first will necessarily remove the rest.

We can extract the principal components easily in **R** using **princomp**:

```
PCAout = princomp(PCAmock)
magplot(PCAmock, asp=1, xlab='x', ylab='y')
arrows(0,0,PCAout$loadings[1,1]*PCAout$sdev[1],PCAout$loadings[1,2]*PCAout$sdev[1],
       col='red',length=0.1,lwd=2)
arrows(0,0,PCAout$loadings[2,1]*PCAout$sdev[2],PCAout$loadings[2,2]*PCAout$sdev[2],
       col='red',length=0.1,lwd=2)
```



Here PCAout\$loadings contains the unit vectors of the principal components and PCAout\$sdev contains

the standard deviation of the data along the relevant component. The total variance in the data then becomes:

```
sum(PCAout$sdev^2)
```

```
## [1] 3.807507
```

And the fraction contained in the first component alone is:

```
PCAout$sdev[1]^2/sum(PCAout$sdev^2)
```

```
##      Comp.1
```

```
## 0.9712847
```

So in this case about 97% of the variance (read that as complexity or information) in the data can be explained by a single component. In this case we might therefore want to describe our data by this single component, which is a vector that combines information from our x and y axes.

Scaling PCA to higher dimensions, a similar approach is used in highly dimensional datasets to find the key few components that contain most of the information. It is also useful to highlight which parameters contain more-or-less the same information.